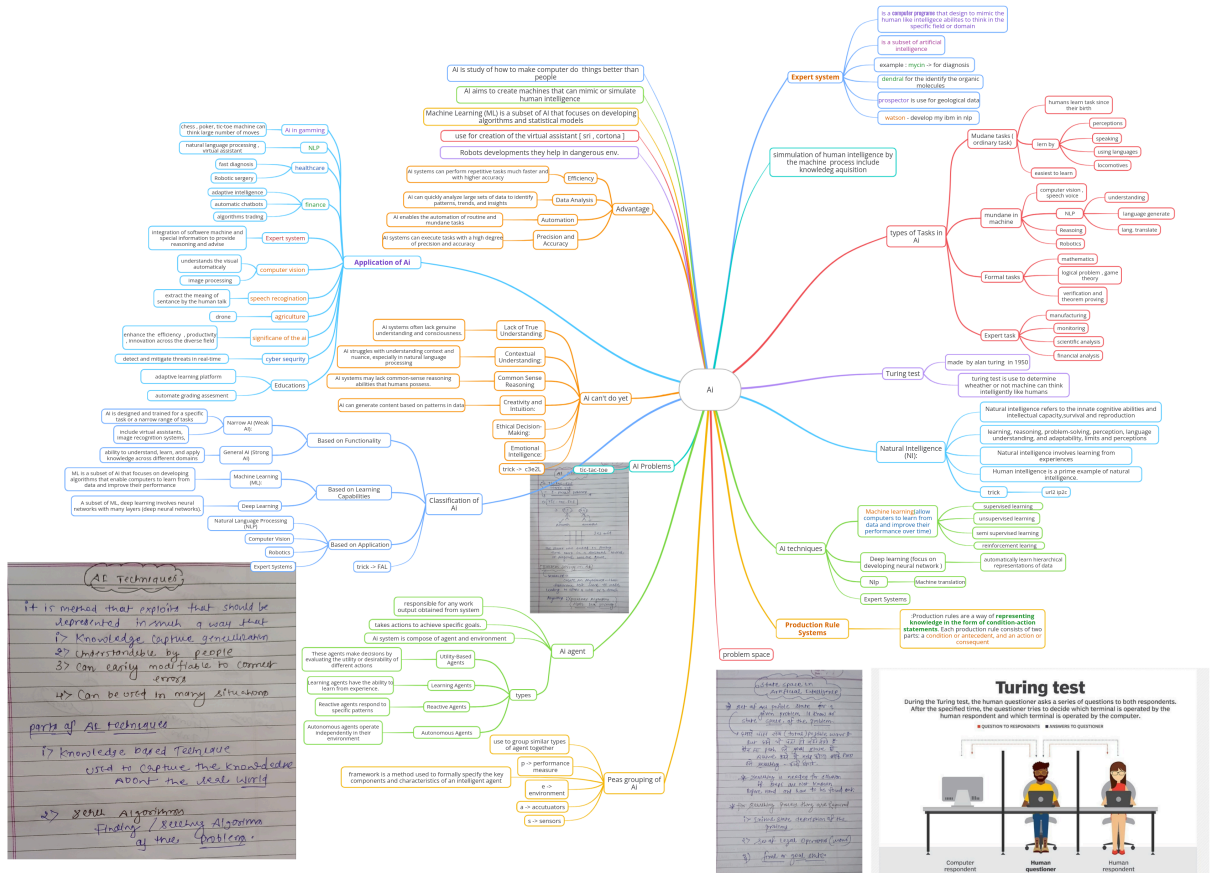
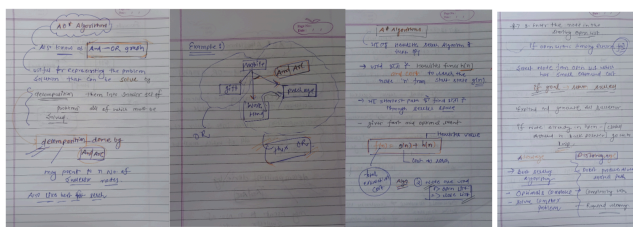
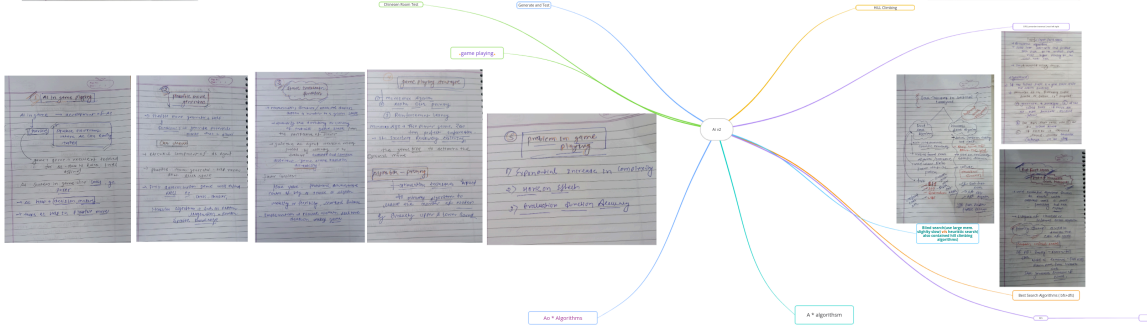
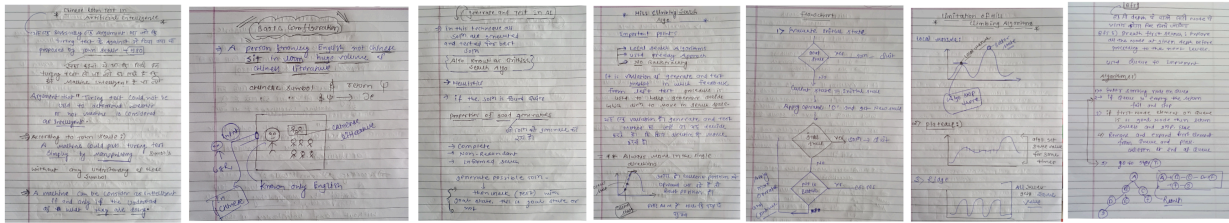
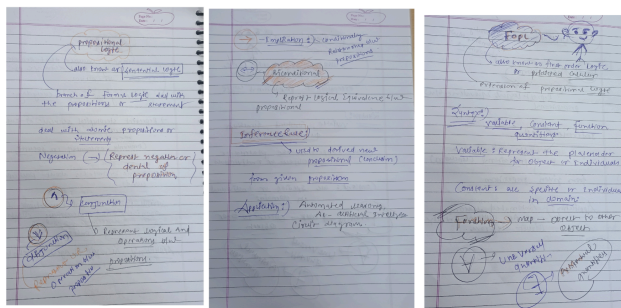
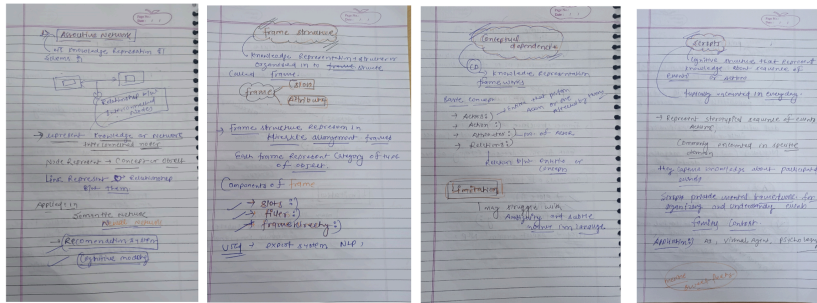


Artificial Intelligence







Module I: Introduction to AI and Problem Representation

1.1 Introduction to Artificial Intelligence

- 1.1.1 Introduction to AI
- 1.1.2 AI and its Importance
- 1.1.3 Application Areas of AI

1.2 Artificial Intelligence Problem

- 1.2.1 AI Problems
- 1.2.2 Tic Tac Toe Problem

- 1.2.3 Water Jug Problem

1.3 AI Representation

- 1.3.1 State Space Representation
 - 1.3.2 Problem Reduction Representation
 - 1.3.3 Production System
 - 1.3.4 Characteristics of Production System
 - 1.3.5 Types of Production System
 - 1.3.6 Recent Industrial Applications of AI
 - 1.3.7 Water Jug Simulation (Python)
 - 1.3.8 Tic Tac Toe Simulation (Python)
 - 1.3.9 8 Puzzle Simulation (Python)
 - 1.3.10 Tower of Hanoi (Prolog)
-

Module II: Heuristic Search Techniques and Game Playing

2.1 Heuristic Search Techniques

- 2.1.1 Heuristic Search Techniques
- 2.1.2 AI and Search Process
- 2.1.3 Brute Force Search
- 2.1.4 Depth First Search (DFS)
- 2.1.5 Breadth First Search (BFS)
- 2.1.6 Time and Space Complexity
- 2.1.7 Heuristic Search
- 2.1.8 Hill Climbing
- 2.1.9 Best First Search
- 2.1.10 A* Algorithm and Beam Search
- 2.1.11 AO Search

- 2.1.12 Constraint Satisfaction
- 2.1.13 Iterative Deepening DFS
- 2.1.14 Bidirectional Search

2.2 Game Playing

- 2.2.1 AI and Game Playing
 - 2.2.2 Move Generator & Static Evaluation
 - 2.2.3 Game Playing Strategies
 - 2.2.4 Problems in Game Playing
-

Module III: Logic and Knowledge Representation

3.1 Knowledge Representation

- 3.1.1 Knowledge Representation
- 3.1.2 Structured Knowledge
- 3.1.3 Associative Networks
- 3.1.4 Frame Structures
- 3.1.5 Conceptual Dependencies
- 3.1.6 Scripts

3.2 Logic Representation

- 3.2.1 Propositional Logic
 - 3.2.2 First Order Predicate Logic (FOPL)
 - 3.2.3 Conversion to Clausal Form
 - 3.2.4 Inference Rules
 - 3.2.5 Unification
 - 3.2.6 Resolution Principle
 - 3.2.7 Naive Bayes Algorithm
-



Module IV: Knowledge Acquisition and Expert System

4.1 Knowledge Acquisition

- 4.1.1 Types of Learning
- 4.1.2 Knowledge Acquisition
- 4.1.3 Early Machine Learning
- 4.1.4 Learning by Induction

4.2 Expert System

- 4.2.1 Introduction to Expert System
 - 4.2.2 Phases of Expert System
 - 4.2.3 Characteristics of Expert System
 - 4.2.4 Case Study
 - 4.2.5 Executive Support System
 - 4.2.6 Decision Support System
 - 4.2.7 Applications of Expert Systems
-



Module V: Robotics and its Applications

5.1 Robotics Basics

- 5.1.1 Robotics and Applications
- 5.1.2 DDD Concept
- 5.1.3 Intelligent Robots
- 5.1.4 Robot Anatomy
- 5.1.5 Laws of Robotics
- 5.1.6 History of Robotics
- 5.1.7 Terminology
- 5.1.8 Accuracy & Repeatability

5.2 Simple Problems in Robotics

- 5.2.1 Robot Specifications & Speed
- 5.2.2 Robot Joints and Links

5.3 Robot Classifications

- 5.3.1 Types of Robots

5.4 Architecture of Robotic Systems

- 5.4.1 System Architecture

5.5 Robot Drive Systems

- 5.5.1 Hydraulic, Pneumatic & Electric Systems

5.6 Core of AI

- 5.6.1 Neural Networks
- 5.6.2 Fuzzy Logic
- 5.6.3 LISP and Prolog
- 5.6.4 Soft Computing Research
- 5.6.5 Knowledge Management
- 5.6.6 Ontology
- 5.6.7 Natural Language Processing (NLP)
- 5.6.8 Genetic Algorithm

Module I: Introduction to AI and Problem Representation

1.1 Introduction to Artificial Intelligence

- 1.1.1 Introduction to AI
- 1.1.2 AI and its Importance

- 1.1.3 Application Areas of AI

1.2 Artificial Intelligence Problem

- 1.2.1 AI Problems
- 1.2.2 Tic Tac Toe Problem
- 1.2.3 Water Jug Problem

1.3 AI Representation

- 1.3.1 State Space Representation
- 1.3.2 Problem Reduction Representation
- 1.3.3 Production System
- 1.3.4 Characteristics of Production System
- 1.3.5 Types of Production System
- 1.3.6 Recent Industrial Applications of AI
- 1.3.7 Water Jug Simulation (Python)
- 1.3.8 Tic Tac Toe Simulation (Python)
- 1.3.9 8 Puzzle Simulation (Python)
- 1.3.10 Tower of Hanoi (Prolog)

1.1 Introduction to AI



1.1.1 Introduction to AI

Definition

Artificial Intelligence (AI) is a branch of computer science that enables machines to perform tasks that require human intelligence such as learning, reasoning, problem-solving, and decision-making.

Key Points

- AI allows machines to **think and act like humans**
 - It works using **data and algorithms**
 - AI systems can **learn from experience**
 - It helps in **automation and efficiency**
 - Used in both **software and hardware systems**
-

Characteristics of AI

- Learning ability
 - Reasoning and decision-making
 - Problem-solving
 - Perception (vision and speech recognition)
 - Adaptability
-

Applications of AI

- Healthcare – disease detection and diagnosis
 - Education – smart learning systems
 - Finance – fraud detection and analysis
 - Transportation – self-driving vehicles
 - E-commerce – product recommendations
 - Agriculture – smart farming and crop prediction
 - Security – face recognition systems
-

Real-Life Applications / Use Cases

- Virtual assistants used for answering queries
 - Recommendation systems for suggesting products or content
 - Self-driving cars for automatic navigation
 - Medical systems for detecting diseases
 - Chatbots for customer support
-

General Use Cases of AI

- Automation of repetitive tasks
 - Data analysis and prediction
 - Decision-making systems
 - Language processing (understanding human language)
 - Image and speech recognition
-

Conclusion

Artificial Intelligence is making machines smarter and more efficient. It is widely used in different fields and helps in solving real-world problems effectively.

1.1.2 AI and its Importance



Definition

Artificial Intelligence (AI) refers to the ability of machines to perform tasks that normally require human intelligence such as learning, reasoning, decision-making, and problem-solving.

Importance of AI

- **Automation of Tasks**
AI helps in automating repetitive and time-consuming tasks, saving human effort.
 - **Increased Efficiency**
AI systems can work faster and more accurately than humans.
 - **Better Decision Making**
AI analyzes large amounts of data to provide accurate and quick decisions.
 - **24/7 Availability**
AI systems can work continuously without breaks.
 - **Reduction in Human Error**
AI reduces mistakes by following programmed rules and learning from data.
 - **Handling Big Data**
AI can process and analyze large datasets efficiently.
 - **Innovation and Advancement**
AI drives new technologies like robotics, self-driving cars, and smart devices.
-

Applications Showing Importance

- Healthcare – early disease detection and treatment
 - Education – personalized learning experience
 - Banking – fraud detection and risk management
 - Business – customer support using chatbots
 - Transportation – smart traffic systems
-

Conclusion

Artificial Intelligence is important because it improves efficiency, reduces human effort, and helps in making better decisions. It plays a key role in modern technology and future development.

.1.3 Application Areas of AI



1.1.3 Application Areas of AI

Definition

Application areas of Artificial Intelligence (AI) refer to the different fields where AI techniques are used to solve problems and perform intelligent tasks.

Major Application Areas of AI

1. Healthcare

- Disease diagnosis and prediction
- Medical imaging analysis
- Drug discovery

2. Education

- Personalized learning systems
- Smart tutoring systems
- Automated grading

3. Finance

- Fraud detection
- Risk assessment
- Algorithmic trading

4. Transportation

- Self-driving cars
- Traffic management systems
- Route optimization

5. E-commerce

- Product recommendation systems
- Customer behavior analysis
- Chatbots for support

6. Agriculture

- Crop prediction
- Smart irrigation systems
- Pest detection

7. Manufacturing

- Robotics and automation
- Quality control
- Predictive maintenance

8. Security

- Face recognition
- Surveillance systems
- Cybersecurity

Real-Life Applications / Use Cases

- Voice assistants for daily tasks
- Online platforms recommending products or movies
- AI-based medical systems detecting diseases
- Smart navigation systems in vehicles
- Automated customer support systems

Conclusion

AI is widely used across multiple fields to improve efficiency, accuracy, and productivity. Its application areas are continuously expanding, making it an essential part of modern technology.

1.2.1 AI Problems



1.2.1 AI Problems

Definition

AI problems are tasks or challenges that require machines to use intelligence such as reasoning, learning, decision-making, and problem-solving to achieve a goal.

Characteristics of AI Problems

- Require **intelligent decision-making**
- Involve **uncertainty and complexity**
- May have **multiple possible solutions**
- Need **search and optimization techniques**
- Often deal with **large amounts of data**

Types of AI Problems

1. Search Problems

- Finding a solution from many possible options
- Example: Path finding, puzzle solving

2. Reasoning Problems

- Making logical conclusions based on given data
- Example: Expert systems

3. Learning Problems

- Learning from data and improving performance
- Example: Machine learning models

4. Planning Problems

- Deciding a sequence of actions to achieve a goal
- Example: Robot navigation

5. Natural Language Processing Problems

- Understanding and processing human language

- Example: Chatbots, translation systems

6. Perception Problems

- Interpreting visual or audio data
 - Example: Image recognition, speech recognition
-

Real-Life Examples of AI Problems

- Finding the shortest route in maps
 - Recognizing faces in images
 - Detecting spam emails
 - Playing games like chess
 - Predicting weather conditions
-

Conclusion

AI problems involve complex tasks that require intelligent solutions. These problems are solved using different AI techniques and are widely used in real-world applications.

1.2.2 Tic Tac Toe Problem



1.2.2 Tic Tac Toe Problem

Definition

The Tic Tac Toe problem is a classic example in AI used to demonstrate how a machine can make decisions using search algorithms and strategies in a game environment.

Problem Description

- It is a **two-player game** played on a **3×3 grid**
- Players take turns marking **X** and **O**
- The goal is to make a **row, column, or diagonal** of the same symbol
- The game ends in a **win or draw**

Representation in AI

- The board is represented as a **state (3×3 matrix)**
- Each move changes the **state of the game**
- The initial state is an **empty board**
- The final state is a **win, lose, or draw**

Key Concepts Used

1. State Space

- All possible configurations of the board

2. Players

- MAX player (tries to win)
- MIN player (tries to block or win)

3. Game Tree

- A tree showing all possible moves from the current state

4. Utility Function

- Assigns values to outcomes:
 - +1 → Win

- 0 → Draw
- 1 → Loss

AI Technique Used

Minimax Algorithm

- AI chooses the best move by **minimizing opponent's chance and maximizing its own chance of winning**
- Explores all possible moves before making a decision

Real-Life Understanding

- Similar to how humans think ahead while playing games
- Used in games like **chess and checkers**

Advantages of Using AI in Tic Tac Toe

- Always plays **optimal moves**
- Helps understand **decision-making process**
- Simple example to learn **game theory**

Conclusion

Tic Tac Toe is a simple yet powerful example to understand AI concepts like search, decision-making, and game strategies using algorithms like Minimax.

1.2.3 Water Jug Problem



1.2.3 Water Jug Problem

Definition

The Water Jug Problem is a classic AI problem used to demonstrate problem-solving using search techniques. It involves measuring an exact quantity of water using two jugs of different capacities.

Problem Description

- Two jugs are given with fixed capacities (e.g., 4 liters and 3 liters)
 - There is an unlimited water supply
 - The goal is to measure a specific quantity (e.g., 2 liters)
 - Operations allowed:
 - Fill a jug
 - Empty a jug
 - Pour water from one jug to another
-

State Representation

- Each state is represented as **(x, y)**
 - x = amount of water in Jug 1
 - y = amount of water in Jug 2
 - Initial state: **(0, 0)**
 - Goal state: desired quantity (e.g., **(2, 0)**)
-

Possible Operations

- Fill Jug 1 or Jug 2
 - Empty Jug 1 or Jug 2
 - Pour water from one jug to another until one is empty or the other is full
-

Solution Approach (Steps Example)

For jugs of 4L and 3L to get 2L:

1. Fill 3L jug → (0, 3)

2. Pour into 4L jug $\rightarrow (3, 0)$
3. Fill 3L jug $\rightarrow (3, 3)$
4. Pour into 4L jug $\rightarrow (4, 2)$
5. Empty 4L jug $\rightarrow (0, 2)$

Goal achieved: 2 liters

AI Techniques Used

- State space search
 - Breadth First Search (BFS)
 - Depth First Search (DFS)
-

Real-Life Use Case

- Measuring exact quantities in cooking or chemical processes
 - Resource allocation problems
 - Planning and decision-making tasks
-

Conclusion

The Water Jug Problem is an important AI example that helps in understanding state space representation and search algorithms for solving real-world problems.

1.3.1 State Space Representation



1.3.1 State Space Representation

Definition

State Space Representation is a method in AI to represent a problem as a set of all possible states, along with the transitions between them, to find a solution.

Key Components

1. State

- A state represents a **particular condition or situation** of the problem
- Example: In a puzzle, arrangement of tiles

2. Initial State

- The **starting point** of the problem
- Example: Empty board or starting configuration

3. Goal State

- The **desired final state** to be achieved
- Example: Target arrangement or solution

4. Operators (Actions)

- Actions that **change one state to another**
- Example: Move, fill, swap

5. State Space

- The **set of all possible states** reachable from the initial state
-

Representation Methods

- **Graph Representation**
States are nodes and actions are edges
 - **Tree Representation**
Shows all possible paths from initial state to goal state
-

Example (Simple Understanding)

- In Tic Tac Toe:

- Each board configuration is a **state**
 - Each move is an **operator**
 - All possible game positions form the **state space**
-

Importance of State Space Representation

- Helps in **systematic problem solving**
 - Makes it easier to apply **search algorithms (BFS, DFS)**
 - Provides a clear **structure of the problem**
 - Helps in finding **optimal solutions**
-

Conclusion

State Space Representation is a fundamental concept in AI that helps in organizing problems and finding solutions efficiently using different search techniques.

1.3.2 Problem Reduction Representation



1.3.2 Problem Reduction Representation

Definition

Problem Reduction Representation is an approach in AI where a complex problem is divided into smaller sub-problems, and each sub-problem is solved separately to reach the final solution.

Basic Idea

- Break a **large problem** into **smaller parts**
 - Solve each sub-problem step by step
 - Combine solutions to get the **final result**
-

Key Components

1. Problem

- The main task that needs to be solved

2. Sub-Problems

- Smaller parts of the main problem

3. Operators

- Actions used to break the problem into sub-problems

4. Goal

- Final solution achieved after solving all sub-problems
-

Representation Structure

AND-OR Graph

- **AND Node** → All sub-problems must be solved
 - **OR Node** → Any one sub-problem can be solved
-

Example (Simple Understanding)

Problem: Prepare for an exam

- Study subjects (AND)
 - Math

- AI
 - DBMS
 - Choose study method (OR)
 - Self-study
 - Coaching
 - Online learning
-

Applications of Problem Reduction

- Game playing (breaking moves into strategies)
 - Planning and scheduling
 - Expert systems
 - Robotics
-

Advantages

- Simplifies complex problems
 - Easy to understand and manage
 - Helps in systematic problem solving
-

Conclusion

Problem Reduction Representation helps in solving complex AI problems by dividing them into smaller manageable parts and using AND-OR graphs to reach the solution efficiently.

1.3.3 Production System



1.3.3 Production System

Definition

A Production System is a rule-based system used in AI where a set of rules is applied to an initial state to transform it into a goal state.

Basic Components

1. Rule Base (Production Rules)

- A set of **IF-THEN** rules
- Example: IF condition → THEN action

2. Working Memory

- Stores the **current state** of the problem
- Contains facts and data

3. Control Strategy

- Determines **which rule to apply** and **when**
 - Guides the execution process
-

Working of Production System

1. Start with an **initial state** in working memory
 2. Match the current state with rules
 3. Apply the suitable rule
 4. Update the state
 5. Repeat until **goal state is reached**
-

Types of Production Systems

1. Monotonic Production System

- Rules do not remove previous facts
- Knowledge only **increases**

2. Non-Monotonic Production System

- Rules can **add or remove facts**

- More flexible

3. Deterministic Production System

- Only **one rule applies** at a time

4. Non-Deterministic Production System

- **Multiple rules** may apply at the same time
-

Example (Simple Understanding)

Rule:

IF temperature is high → THEN turn on fan

- Current state: Temperature is high
 - Action: Fan is turned on
-

Applications of Production System

- Expert systems
 - Game playing
 - Decision-making systems
 - Problem-solving tasks
-

Advantages

- Simple and easy to understand
 - Flexible and modular
 - Useful for rule-based reasoning
-

Conclusion

Production System is an important AI technique that uses rules and control strategies to solve problems step by step and reach the desired goal.

1.3.4 Characteristics of Production System



1.3.4 Characteristics of Production System

Definition

Characteristics of a Production System describe the important features that define how rules are applied and how the system behaves while solving a problem.

Main Characteristics

1. Modularity

- Knowledge is represented in the form of **independent rules**
- Easy to add, remove, or modify rules without affecting others

2. Flexibility

- Can handle **different types of problems**
- Rules can be changed according to requirements

3. Separation of Knowledge and Control

- **Rule base** stores knowledge
- **Control strategy** decides how rules are applied

4. Transparency

- Easy to understand and trace **which rule is applied**
- Helps in debugging and explanation

5. Incrementality

- New rules can be added **gradually**
- System can grow over time

6. Control Strategy

- Determines the **order of rule execution**
- Important for efficiency and correctness

7. Conflict Resolution

- When multiple rules are applicable, the system decides **which rule to apply**
-

Advantages

- Easy to design and implement
 - Suitable for complex problem solving
 - Reusable and maintainable
-

Conclusion

Production Systems are powerful because of their modular and flexible nature. Their characteristics make them suitable for building intelligent systems that can solve problems efficiently.

1.3.5 Types of Production System



1.3.5 Types of Production System

Definition

Types of Production System refer to the different categories based on how rules are applied and how the system behaves during problem solving.

Main Types of Production System

1. Monotonic Production System

- Rules **do not remove or change existing facts**
- Knowledge only **increases**
- Once applied, rules do not affect previous results

Example: Mathematical problem solving

2. Non-Monotonic Production System

- Rules can **add, remove, or modify facts**
- Allows **updating of knowledge**
- More flexible and realistic

Example: Real-world decision making

3. Deterministic Production System

- Only **one rule is applicable** at a time
- Output is **predictable**
- Same input always gives same result

Example: Simple calculations

4. Non-Deterministic Production System

- **Multiple rules may apply** at the same time
- System must choose one using control strategy
- Output may vary

Example: Game playing (like chess)

Comparison (Quick Revision Table)

Type	Feature
Monotonic	No change in previous facts
Non-Monotonic	Facts can be changed
Deterministic	One rule applies
Non-Deterministic	Multiple rules apply

Conclusion

Different types of production systems are used based on the problem requirements. They help in applying rules effectively and solving problems in AI systems.

1.3.6 Recent Industrial Applications of AI



1.3.6 Recent Industrial Applications of AI

Definition

Recent industrial applications of AI refer to the use of Artificial Intelligence technologies in modern industries to improve efficiency, automation, and decision-making.

Major Industrial Applications of AI

1. Manufacturing Industry

- Use of **robots for automation**
 - **Quality control** using image recognition
 - **Predictive maintenance** of machines
-

2. Healthcare Industry

- Disease diagnosis using AI systems
 - Medical image analysis (X-rays, MRI)
 - Drug discovery and research
-

3. Banking and Finance

- Fraud detection systems
 - Risk assessment and credit scoring
 - Automated trading systems
-

4. Retail and E-commerce

- Product recommendation systems
 - Customer behavior analysis
 - Chatbots for customer service
-

5. Transportation and Logistics

- Self-driving vehicles
- Route optimization
- Traffic prediction systems

6. Agriculture Industry

- Smart farming using AI
- Crop and soil analysis
- Automated irrigation systems

7. IT and Software Industry

- AI-powered coding assistants
- Bug detection and testing
- Cybersecurity systems

Real-Life Use Cases

- Online platforms recommending products
- AI chatbots handling customer queries
- Smart assistants performing tasks
- AI detecting fraud in banking transactions
- Robots working in factories

Advantages of AI in Industry

- Increased productivity
- Reduced human effort
- Improved accuracy
- Faster decision-making
- Cost reduction

Conclusion

AI is transforming modern industries by making processes smarter, faster, and more efficient. Its applications are continuously growing and shaping the future of industrial development.



1.3.7 Water Jug Simulation (Python)

Definition

Water Jug Simulation is the implementation of the Water Jug Problem using Python programming to find the solution using AI search techniques.

Problem Statement

- Two jugs with capacities **4L and 3L**
- Unlimited water supply
- Goal: Measure **2 liters of water**

Approach

- Represent each state as **(x, y)**
 - x = water in Jug 1
 - y = water in Jug 2
- Use **state space search (BFS/DFS)** to reach goal state

```
from collections import deque
def water_jug_bfs(jug1, jug2, target):
    visited = set()
    queue = deque([(0, 0)]) # initial state
```

```
    while queue:
        x, y = queue.popleft()

        # Check if goal is reached
        if x == target or y == target:
            print("Goal reached:", (x, y))
            return

        if (x, y) in visited:
            continue

        visited.add((x, y))

        # Possible operations
        queue.append((jug1, y)) # Fill Jug1
        queue.append((x, jug2)) # Fill Jug2
        queue.append((0, y))    # Empty Jug1
```

```
queue.append((x, 0))    # Empty Jug2

# Pour Jug1 -> Jug2
transfer = min(x, jug2 - y)
queue.append((x - transfer, y + transfer))

# Pour Jug2 -> Jug1
transfer = min(y, jug1 - x)
queue.append((x + transfer, y - transfer))

print("No solution found")
```

Example

water_jug_bfs(4, 3, 2)

Explanation

- Start from **(0,0)**
- Apply all possible operations
- Use **queue (BFS)** to explore states
- Stop when target (2 liters) is reached

Applications

- Problem-solving using AI
- Understanding search algorithms
- Resource allocation problems

1.3.8 Tic Tac Toe Simulation (Python)



1.3.8 Tic Tac Toe Simulation (Python)

Definition

Tic Tac Toe Simulation is the implementation of the Tic Tac Toe game using Python, where AI can make decisions using logic or algorithms like Minimax.

Problem Description

- A **two-player game** played on a **3×3 grid**
- Players use **X and O**
- Goal: Get **3 symbols in a row, column, or diagonal**

Approach

- Represent board as a **list of 9 elements**
- Take input from user and update board
- Check for **win, loss, or draw** after each move

Python Program

```
def print_board(board):
    for i in range(0, 9, 3):
        print(board[i], "|", board[i+1], "|", board[i+2])
        print("-"*9)

def check_winner(board, player):
    win_positions = [
        [0, 1, 2], [3, 4, 5], [6, 7, 8], # rows
        [0, 3, 6], [1, 4, 7], [2, 5, 8], # columns
        [0, 4, 8], [2, 4, 6] # diagonals
    ]
    for pos in win_positions:
        if board[pos[0]] == board[pos[1]] == board[pos[2]] == player:
            return True
    return False

def tic_tac_toe():
    board = [" " for _ in range(9)]
    current_player = "X"

    for turn in range(9):
        print_board(board)
        move = int(input(f"Player {current_player}, enter position (0-8): "))
```

```
if board[move] == " ":
    board[move] = current_player
else:
    print("Invalid move! Try again.")
    continue

if check_winner(board, current_player):
    print_board(board)
    print(f"Player {current_player} wins!")
    return

current_player = "O" if current_player == "X" else "X"

print_board(board)
print("It's a draw!")

# Run the game
tic_tac_toe()
```

Explanation

- Board is stored as a **list**
- Players take turns to input positions
- Program checks **winning conditions**
- Game ends with **win or draw**

Applications

- Understanding game logic in AI
- Learning decision-making strategies
- Basis for advanced AI algorithms like Minimax

Conclusion

Tic Tac Toe Simulation in Python helps in understanding AI concepts like game playing, decision-making, and strategy implementation in a simple and practical way.

1.3.9 8 Puzzle Simulation (Python)



1.3.9 8 Puzzle Simulation (Python)

Definition

8 Puzzle Simulation is the implementation of the 8-puzzle problem using Python, where tiles are moved to reach a goal state using AI search techniques.

Problem Description

- A **3×3 grid** with **8 numbered tiles** and **1 empty space (0)**
- Tiles can move **up, down, left, right**
- Goal: Arrange tiles in **correct order**

Example Goal State:

```
1 2 3
4 5 6
7 8 0
```

Approach

- Represent puzzle as a **2D list (matrix)**
- Use **state space search (BFS)**
- Generate all possible moves from current state
- Stop when **goal state is reached**

Python Program (Using BFS)

```
from collections import deque

# Goal state
goal = [[1, 2, 3], [4, 5, 6], [7, 8, 0]]

# Directions for movement
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

def find_zero(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
```

```

return i, j

def is_goal(state):
    return state == goal

def get_next_states(state):
    x, y = find_zero(state)
    next_states = []

    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            next_states.append(new_state)

    return next_states

def bfs(start):
    visited = set()
    queue = deque([(start, [])])

    while queue:
        state, path = queue.popleft()

        if is_goal(state):
            print("Solution found!")
            for step in path + [state]:
                for row in step:
                    print(row)
                print()
            return

        state_tuple = tuple(map(tuple, state))
        if state_tuple in visited:
            continue

        visited.add(state_tuple)

        for next_state in get_next_states(state):
            queue.append((next_state, path + [state]))

    print("No solution found")

# Initial state
start = [[1, 2, 3],
         [4, 0, 6],
         [7, 5, 8]]

bfs(start)

```

Explanation

- The puzzle is represented as a **matrix**
 - Empty space (0) is moved in all valid directions
 - **BFS explores all states level by level**
 - Stops when goal configuration is reached
-

Applications

- AI problem-solving techniques
 - Game solving algorithms
 - Pathfinding and search problems
-

Conclusion

8 Puzzle Simulation helps in understanding state space, search algorithms, and problem-solving techniques in AI using Python.

1.3.10 Tower of Hanoi (Prolog)



1.3.10 Tower of Hanoi (Prolog)

Definition

Tower of Hanoi is a classical problem in AI used to demonstrate recursion and problem-solving. It involves moving disks from one rod to another following specific rules.

Problem Description

- Three rods: **Source (A), Auxiliary (B), Destination (C)**
- A number of disks of different sizes
- Rules:
 - Only **one disk** can be moved at a time
 - A **larger disk cannot be placed on a smaller disk**
 - All disks must be moved from **Source to Destination**

Approach

- Use **recursion** to break the problem into smaller sub-problems
- Move **n-1 disks** to auxiliary rod
- Move the largest disk
- Move **n-1 disks** to destination rod

Prolog Program

```
% Tower of Hanoi Program

move(1, Source, Destination, _) :-
    write('Move disk from '), write(Source),
    write(' to '), write(Destination), nl.

move(N, Source, Destination, Auxiliary) :-
    N > 1,
    M is N - 1,
    move(M, Source, Auxiliary, Destination),
    move(1, Source, Destination, Auxiliary),
    move(M, Auxiliary, Destination, Source).
```

How to Run

```
?- move(3, a, c, b).
```

Output (Example)

```
Move disk from a to c  
Move disk from a to b  
Move disk from c to b  
Move disk from a to c  
Move disk from b to a  
Move disk from b to c  
Move disk from a to c
```

Explanation

- The problem is solved using **recursion**
- Breaks into smaller problems until only **one disk** remains
- Then builds the solution step by step

Applications

- Understanding recursion
- Problem-solving in AI
- Used in algorithm design

Conclusion

Tower of Hanoi in Prolog demonstrates recursive problem-solving and helps in understanding how complex problems can be solved step by step using logical programming.

Module II: Heuristic Search Techniques and Game Playing

2.1.1 Heuristic Search Techniques (AI)



2.1.1 Heuristic Search Techniques (AI)

Heuristic search techniques are **intelligent search methods** used in Artificial Intelligence to solve problems faster by using **domain-specific knowledge (heuristics)** instead of blindly exploring all possibilities.



What is a Heuristic?

A **heuristic** is a rule, strategy, or shortcut that helps guide the search toward the **goal state more efficiently**.

👉 Instead of checking all paths (like in brute-force search), heuristic methods:

- Estimate which path is **best**
 - Reduce **time and space complexity**
-



Key Idea

Heuristic search uses an **evaluation function**:

$$f(n) = g(n) + h(n)$$

$$f(n) = g(n) + h(n)$$

- **$g(n)$** → cost from start node to current node
 - **$h(n)$** → estimated cost from current node to goal (heuristic)
 - **$f(n)$** → total estimated cost
-



Types of Heuristic Search Techniques

1. Generate and Test

- Generate a possible solution and test if it satisfies the goal.
- Repeat until solution found.

👉 Simple but inefficient.

2. Hill Climbing Algorithm

- Moves in the direction of increasing value (better state).
- Like climbing a hill to reach the top (goal).

Types:

- Simple Hill Climbing
- Steepest-Ascent Hill Climbing
- Stochastic Hill Climbing

⚠ Problems:

- Local maxima
- Plateaus
- Ridge problems

3. Best-First Search

- Expands the most promising node based on heuristic value.

👉 Uses:

$$f(n) = h(n)$$

$$f(n) = h(n)$$

4. Greedy Best-First Search

- Chooses the node closest to the goal (based only on $h(n)$).

✓ Fast

✗ Not always optimal

5. A* (A-Star) Algorithm

- Most popular heuristic search technique.

Uses:

$$f(n) = g(n) + h(n)$$

$$f(n) = g(n) + h(n)$$

✓ Complete (always finds solution if exists)

✓ Optimal (if heuristic is admissible)

6. Beam Search

- Like BFS but keeps only a limited number of best nodes (beam width).

✓ Memory efficient

✗ May miss optimal solution

7. Simulated Annealing

- Inspired by metallurgy process.
 - Sometimes allows bad moves to escape local maxima.
 - ✓ Avoids local optima
 - ✓ Good for large problems
-

Advantages of Heuristic Search

- Faster than uninformed search
 - Reduces search space
 - Practical for real-world problems
-

Disadvantages

- May not give optimal solution
 - Depends on quality of heuristic
 - Designing good heuristic is difficult
-

Real-World Applications

- Pathfinding (Google Maps, GPS)
 - Game AI (Chess, Tic-Tac-Toe)
 - Robotics navigation
 - Optimization problems
-

Simple Example

If you're going from Patna to Delhi:

- Blind search → try all routes
- Heuristic search → choose route with **shortest estimated distance**

2.1.2 AI and Search Process



2.1.2 AI and Search Process

In Artificial Intelligence, the **search process** is a fundamental technique used to solve problems by exploring possible solutions step by step until the **goal state** is reached.



What is Search in AI?

Search is the process of:

- Exploring different possible **states**
- Finding a sequence of actions from **initial state** → **goal state**



It is used when the solution is not directly known.



Components of Search Process

1. Initial State

- Starting point of the problem



Example: Starting position in a maze

2. State Space

- Set of all possible states reachable from the initial state



Example: All possible board configurations in a puzzle

3. Operators (Actions)

- Rules that define how to move from one state to another



Example: Move left, right, up, down

4. Goal State

- Desired final state



Example: Destination reached

5. Path

- Sequence of states from initial to goal
-

6. Cost Function

- Assigns cost to each path

👉 Example: Distance, time, energy

Search Tree Representation

- Root node → Initial state
 - Branches → Possible actions
 - Nodes → States
 - Leaf nodes → End states
-

Types of Search in AI

1. Uninformed Search (Blind Search)

- No extra knowledge used
- Explores blindly

Examples:

- Breadth-First Search (BFS)
 - Depth-First Search (DFS)
 - Uniform Cost Search
-

2. Informed Search (Heuristic Search)

- Uses domain knowledge (heuristics)

Examples:

- Greedy Search
 - A* Algorithm
-

General Steps in AI Search Process

1. Define the problem
2. Identify initial state
3. Define goal state
4. Generate possible states

5. Apply search strategy
 6. Evaluate paths
 7. Reach goal or fail
-

Example: 8-Puzzle Problem

- Initial: Random arrangement
 - Goal: Ordered tiles
 - Operators: Move tiles
 - Search: Find shortest moves
-

Properties of Search Algorithms

- **Completeness** → Will it find a solution?
 - **Optimality** → Is the solution best?
 - **Time Complexity** → Speed
 - **Space Complexity** → Memory usage
-

Importance of Search in AI

- Core of problem-solving
 - Used in games, robotics, navigation
 - Helps in decision-making systems
-

Real-Life Example

Google Maps:

- Initial state → Your location
 - Goal state → Destination
 - Search process → Finds best route
-

Short Exam Definition

 **AI Search Process:**

"A method of exploring possible states and actions to find a path from an initial state to a goal state."

2.1.3 Brute Force Search (AI)



2.1.3 Brute Force Search (AI)

Brute Force Search is the simplest problem-solving technique in Artificial Intelligence where **all possible solutions are generated and checked** until the correct one is found.

👉 It is also called **Blind Search** because it does not use any domain knowledge or heuristics.



Definition

👉 **Brute Force Search:**

"A method that systematically explores the entire search space without using any guidance to find the goal state."



Key Characteristics

- No intelligence or heuristic used
- Explores **all possible states**
- Guarantees finding a solution (if it exists)
- Very **time-consuming and inefficient**



How It Works

1. Start from the **initial state**
2. Generate all possible states
3. Check each state one by one
4. Stop when **goal state is found**



Example

◆ Password Cracking

Trying all combinations:

- 0000 → 0001 → 0002 → ... → 9999

👉 Eventually finds correct password but takes time.

◆ 8-Puzzle Problem

- Try all possible moves until goal state is reached

Types of Brute Force Search

1. Breadth-First Search (BFS)

- Explores level by level
- Finds shortest path

2. Depth-First Search (DFS)

- Explores deeply first
- Uses less memory

Advantages

- ✓ Simple to implement
- ✓ Guaranteed to find solution
- ✓ No need for domain knowledge

Disadvantages

- ✗ Very slow for large problems
- ✗ High time complexity
- ✗ May consume large memory (BFS)
- ✗ Not practical for real-world large-scale problems

Comparison with Heuristic Search

Feature	Brute Force Search	Heuristic Search
Knowledge Used	✗ No	✓ Yes
Speed	✗ Slow	✓ Faster
Efficiency	✗ Low	✓ High
Optimality	Sometimes	Often

Real-Life Analogy

Searching for a book in a library:

- Brute force → Check **every book one by one**
- Smart search → Use **catalog system**

2.1.4 Depth First Search (DFS)



2.1.4 Depth First Search (DFS)

Depth First Search (DFS) is a **brute force search technique** in Artificial Intelligence that explores a search tree by going **as deep as possible along one branch before backtracking**.



Definition



Depth First Search (DFS):

"A search algorithm that explores the deepest nodes of a tree or graph first before moving to other branches."



Key Idea

- Go **deep first**, then backtrack
- Uses **Stack (LIFO)** principle

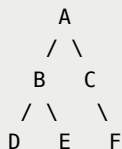


Working of DFS

1. Start from the **initial node (root)**
2. Visit a node
3. Move to one of its **child nodes**
4. Continue until:
 - Goal is found ✓
 - No more children → **Backtrack** 
5. Repeat for other branches



Example (Tree Traversal)



DFS Traversal:

A → B → D → E → C → F

Algorithm (Steps)

1. Push root node into stack
2. Pop node and visit it
3. Push its children into stack
4. Repeat until stack is empty or goal found

Pseudocode

```
DFS(node):  
    if node is goal:  
        return success  
    mark node as visited  
    for each child of node:  
        DFS(child)
```

Advantages

- ✓ Uses less memory than BFS
- ✓ Simple to implement
- ✓ Good for deep solutions

Disadvantages

- ✗ Not guaranteed to find shortest path
- ✗ Can get stuck in infinite loops
- ✗ May miss optimal solution

Time & Space Complexity

- **Time Complexity:** $O(b^d)$
- **Space Complexity:** $O(b \times d)$

Where:

- **b** = branching factor
- **d** = depth

DFS vs BFS

Feature	DFS	BFS
Approach	Depth first	Level-wise
Data Structure	Stack	Queue
Memory	Low	High
Optimal	✗ No	✓ Yes



Real-Life Example



Solving a maze:

- DFS → Follow one path till dead end, then backtrack

2.1.5 Breadth First Search (BFS)



2.1.5 Breadth First Search (BFS)

Breadth First Search (BFS) is a **brute force (uninformed) search technique** in Artificial Intelligence that explores a tree or graph **level by level**, starting from the root node.



Definition



Breadth First Search (BFS):

"A search algorithm that explores all nodes at the present depth level before moving to the next level."



Key Idea

- Explore **neighbors first**, then go deeper
- Uses **Queue (FIFO – First In First Out)**

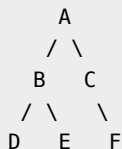


Working of BFS

1. Start from the **initial node (root)**
2. Visit the node
3. Add all its **child nodes to a queue**
4. Remove node from queue and visit it
5. Repeat until:
 - Goal is found ✓
 - Queue becomes empty ✗



Example (Tree Traversal)



BFS Traversal:

A → B → C → D → E → F

Algorithm (Steps)

1. Insert root node into queue
2. Remove node from queue
3. Visit node
4. Add its children to queue
5. Repeat

Pseudocode

```
BFS(node):  
    create queue  
    enqueue(node)  
  
    while queue is not empty:  
        current = dequeue()  
        visit(current)  
  
        for each child of current:  
            enqueue(child)
```

Advantages

- ✓ Finds **shortest path** (in unweighted graphs)
- ✓ Complete (guaranteed to find solution)
- ✓ Simple and systematic

Disadvantages

- ✗ High memory usage (stores all nodes at a level)
- ✗ Slow for deep trees

Time & Space Complexity

- **Time Complexity:** $O(b^d)$
- **Space Complexity:** $O(b^d)$

Where:

- **b** = branching factor

- **d** = depth

BFS vs DFS

Feature	BFS	DFS
Approach	Level-wise	Depth-wise
Data Structure	Queue	Stack
Memory	High	Low
Optimal	✓ Yes	✗ No

Real-Life Example

👉 Social networks (like finding shortest connection path):

- BFS finds **minimum number of connections**

Short Exam Definition

2.1.6 Time and Space Complexity (AI Search Algorithms)



2.1.6 Time and Space Complexity (AI Search Algorithms)

In Artificial Intelligence, **Time and Space Complexity** are used to measure the **efficiency of search algorithms**.



1. Time Complexity

👉 Definition:

Time complexity represents the **total number of nodes generated or explored** during the search process.

- It tells us how **fast or slow** an algorithm is
- Usually expressed using **Big-O notation**



2. Space Complexity

👉 Definition:

Space complexity represents the **amount of memory required** to store nodes during the search.

- Includes:
 - Nodes in memory
 - Data structures (queue, stack, etc.)



Important Terms

- **b (Branching Factor):** Number of children per node
- **d (Depth):** Depth of the goal node
- **m (Max Depth):** Maximum depth of search tree



Complexity of Common Search Algorithms

Algorithm	Time Complexity	Space Complexity	Complete	Optimal
Breadth-First Search	$O(b^d)$	$O(b^d)$	✓ Yes	✓ Yes
Depth-First Search	$O(b^m)$	$O(b \times m)$	✗ No	✗ No

Algorithm	Time Complexity	Space Complexity	Complete	Optimal
Uniform Cost Search	$O(b^d)$	$O(b^d)$	✓ Yes	✓ Yes
Greedy Search	$O(b^m)$	$O(b^m)$	✗ No	✗ No
A* Algorithm	$O(b^d)$	$O(b^d)$	✓ Yes	✓ Yes

Explanation

♦ Breadth-First Search (BFS)

- Explores all nodes level by level
- Needs to store entire level → **high memory**

♦ Depth-First Search (DFS)

- Goes deep into tree
- Stores only current path → **low memory**

♦ Uniform Cost Search

- Expands lowest cost node first
- Similar complexity to BFS

♦ Greedy Search

- Uses heuristic only
- Faster but not optimal

♦ A* Search

- Combines cost + heuristic
- Efficient and optimal (if heuristic is good)

Key Observations

- BFS → **High time + high space**
- DFS → **Low space but may be slow**
- A* → **Best balance (time + optimality)**

Simple Understanding

👉 If a tree has:

- $b = 3$ (each node has 3 children)
- $d = 4$

Then:

- Total nodes $\approx 3^4 = 81$ nodes

So complexity = $O(b^d)$

Why It Matters

- Helps choose the **right algorithm**
- Important for **large-scale problems**
- Affects performance in:
 - Games
 - Navigation systems
 - AI models

2.1.7 Heuristic Search (AI)



2.1.7 Heuristic Search (AI)

Heuristic Search is an **informed search technique** in Artificial Intelligence that uses **additional knowledge (heuristics)** to find solutions more efficiently than blind search methods.



Definition



Heuristic Search:

"A search technique that uses domain-specific knowledge to guide the search process toward the goal state more efficiently."



What is a Heuristic?

A **heuristic ($h(n)$)** is an estimate of the **cost from the current node to the goal**.



It helps answer:

"Which path looks most promising?"



Evaluation Function

Most heuristic algorithms use:

- **$g(n)$** → Actual cost from start to current node
- **$h(n)$** → Estimated cost to goal
- **$f(n)$** → Total estimated cost



Key Characteristics

- Uses **intelligence / prior knowledge**
- Reduces **search space**
- Faster than brute force methods
- May or may not give optimal solution



Types of Heuristic Search

1. Greedy Best-First Search

- Uses only heuristic value:

$$f(n) = h(n) \quad f(n) = h(n)$$

$$f(n)=h(n)$$

✓ Fast

✗ Not always optimal

2. A* (A-Star) Algorithm

- Combines cost + heuristic:

✓ Complete

✓ Optimal (if heuristic is admissible)

3. Hill Climbing

- Moves toward better states step by step

✗ Can get stuck in local maxima

4. Beam Search

- Explores limited best nodes

✓ Memory efficient



Heuristic vs Brute Force

Feature	Heuristic Search	Brute Force Search
Knowledge Used	✓ Yes	✗ No
Speed	✓ Fast	✗ Slow
Efficiency	✓ High	✗ Low
Optimal Solution	Sometimes	Sometimes



Example

👉 Pathfinding (Google Maps):

- Heuristic → Straight-line distance to destination
 - Helps choose **shortest route faster**
-



Advantages

✓ Faster and efficient

✓ Reduces unnecessary exploration

✓ Useful in real-world problems

Disadvantages

-  Depends on quality of heuristic
 -  May give wrong or suboptimal solution
 -  Designing heuristic is difficult
-

Applications

- Navigation systems (GPS)
- Game AI (Chess, Tic-Tac-Toe)
- Robotics
- Optimization problems

2.1.8 Hill Climbing (AI)



2.1.8 Hill Climbing (AI)

Hill Climbing is a **heuristic search algorithm** that continuously moves toward a **better state (higher value)** until it reaches the **best possible solution**.

👉 It is called "hill climbing" because it is like **climbing a hill step by step to reach the top (optimal solution)**.



Definition

👉 **Hill Climbing:**

"A heuristic search technique that iteratively moves to a neighboring state with a better value until no further improvement is possible."



Key Idea

- Start with an initial solution
 - Move to a **better neighboring state**
 - Stop when no better neighbor exists
-



Working Steps

1. Start with an **initial state**
 2. Evaluate neighboring states
 3. Move to the **best neighbor**
 4. Repeat until:
 - Goal is reached ✅
 - No improvement possible ❌
-



Example

👉 Imagine climbing a hill:

- You take steps upward
 - Stop when you reach the **peak**
-



Types of Hill Climbing

1. Simple Hill Climbing

- Picks the first better neighbor
-

2. Steepest-Ascent Hill Climbing

- Chooses the **best among all neighbors**
-

3. Stochastic Hill Climbing

- Chooses a random better neighbor
-

Problems in Hill Climbing

1. Local Maximum

- Stops at a peak that is not the highest
-

2. Plateau

- Flat area → no direction to move
-

3. Ridge Problem

- Difficult to move in correct direction
-

Advantages

- ✓ Simple and fast
 - ✓ Uses less memory
 - ✓ Good for optimization problems
-

Disadvantages

-  Can get stuck in local maxima
 -  Not guaranteed to find optimal solution
 -  No backtracking
-

Time & Space Complexity

- **Time Complexity:** Depends on problem (generally low)
 - **Space Complexity:** $O(1)$ (very low memory)
-

2.1.9 Best First Search (AI)



2.1.9 Best First Search (AI)

Best First Search is a **heuristic search algorithm** that selects and expands the **most promising node** based on a given evaluation function.

👉 It always chooses the node that appears to be **closest to the goal**.



Definition

👉 **Best First Search:**

"A search algorithm that expands the node which is estimated to be closest to the goal using a heuristic function."



Key Idea

- Use a **heuristic function $h(n)$**
 - Always pick the **best node first**
 - Uses a **priority queue** (instead of stack/queue)
-



Evaluation Function

- **$h(n)$** → Estimated cost from current node to goal

👉 Node with **lowest $h(n)$** is selected first.



Working Steps

1. Start with initial node
 2. Evaluate using heuristic function
 3. Select node with **lowest $h(n)$**
 4. Expand that node
 5. Repeat until goal is reached
-



Example

Imagine finding shortest path:

- Choose node that looks **closest to destination**
 - Ignore other paths for now
-

Types of Best First Search

1. Greedy Best First Search

- Uses only heuristic → **fast but not optimal**

2. A* Search

- Uses both cost and heuristic → **optimal**

Advantages

- ✓ Faster than blind search
- ✓ Uses domain knowledge
- ✓ Efficient in many cases

Disadvantages

- ✗ May not find optimal solution
- ✗ Depends on heuristic accuracy
- ✗ Can get stuck in wrong path

Time & Space Complexity

- **Time Complexity:** $O(b^m)$
- **Space Complexity:** $O(b^m)$

Where:

- **b** = branching factor
- **m** = maximum depth

Comparison

Feature	Best First Search	BFS / DFS
Type	Heuristic	Uninformed
Speed	Faster	Slower
Optimal	✗ Not always	BFS ✓
Memory	High	Varies

Real-Life Example

👉 Google Maps:

- Chooses route that **appears closest to destination first**

2.1.10 A* Algorithm and Beam Search



2.1.10 A* Algorithm and Beam Search (AI)

This topic covers two important **heuristic search techniques** used to solve complex problems efficiently.



1. A* (A-Star) Algorithm



Definition

👉 A* Algorithm:

"A heuristic search algorithm that finds the optimal path by combining actual cost and estimated cost."



Evaluation Function

- **$g(n)$** → Cost from start to current node
- **$h(n)$** → Estimated cost to goal
- **$f(n)$** → Total estimated cost

👉 Node with **lowest $f(n)$** is expanded first.



Working Steps

1. Start from initial node
 2. Calculate **$f(n)$** for all neighbors
 3. Select node with lowest **$f(n)$**
 4. Expand and repeat
 5. Stop when goal is reached
-



Properties

- ✓ **Complete** (always finds solution if exists)
 - ✓ **Optimal** (if heuristic is admissible)
 - ✓ Efficient and widely used
-



Example

👉 Pathfinding (maps, GPS):

- Finds **shortest route** considering distance + estimate

Disadvantages

-  High memory usage
 -  Performance depends on heuristic
-

2. Beam Search

Definition

Beam Search:

"A heuristic search algorithm that explores only a limited number of best nodes at each level."

Key Idea

- Similar to BFS
- But keeps only **k best nodes (beam width)**

 Reduces memory and computation

Working Steps

1. Start from initial node
 2. Generate all children
 3. Select top **k best nodes** (based on heuristic)
 4. Discard others
 5. Repeat until goal is found
-

Example

If beam width **k = 2**:

- Only best 2 nodes are explored at each level
-

Advantages

- ✓ Memory efficient
 - ✓ Faster than BFS
 - ✓ Useful for large problems
-

Disadvantages

-  May miss optimal solution
-  Not complete
-  Depends on beam width

2.1.11 AO* Search (AND-OR Search Algorithm)



2.1.11 AO* Search (AND-OR Search Algorithm)

AO* Search is a **heuristic search algorithm** used for problems that can be represented using an **AND-OR graph**, where solving a problem may require solving **multiple subproblems together (AND)** or **choosing one among alternatives (OR)**.



Definition

👉 AO* Search:

"A heuristic search algorithm that finds the optimal solution in an AND-OR graph by considering both AND and OR conditions."



What is an AND-OR Graph?

♦ OR Node

- Choose **any one** child to reach the goal
👉 Example: Choose one route to reach destination

♦ AND Node

- Must satisfy **all children**
👉 Example: Complete all tasks to finish a project



Key Idea

- Combines **problem decomposition + heuristic search**
- Finds **optimal solution graph (not just path)**
- Updates cost dynamically



Cost Calculation

- OR Node:**
👉 Choose minimum cost path
- AND Node:**
👉 Sum of all child costs

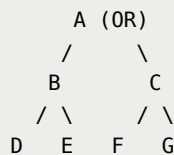


Working Steps

1. Start from initial node

2. Expand nodes using heuristic
3. Identify AND / OR relationships
4. Compute cost:
 - OR → minimum
 - AND → sum
5. Mark best path
6. Continue until goal is reached

Example (Simple Idea)



- A is OR → choose best between B and C
- B is AND → must solve D and E

Advantages

- ✓ Finds optimal solution for complex problems
- ✓ Handles problem decomposition
- ✓ Efficient for structured problems

Disadvantages

- ✗ Complex to implement
- ✗ Needs good heuristic
- ✗ Higher computation overhead

Applications

- Problem reduction systems
- Game playing
- Planning and decision-making

- Expert systems

AO* vs A*

Feature	AO* Search	A* Algorithm
Graph Type	AND-OR Graph	Normal Graph
Solution	Solution Graph	Single Path
Complexity	Higher	Moderate
Use Case	Decomposition problems	Pathfinding

Real-Life Example

 Project completion:

- Some tasks require **all steps (AND)**
- Some allow **choice (OR)**

2.1.12 Constraint Satisfaction (CSP) in AI



2.1.12 Constraint Satisfaction (CSP) in AI

Constraint Satisfaction Problems (CSPs) are a class of problems in Artificial Intelligence where the goal is to **find values for variables that satisfy a set of constraints**.



Definition



Constraint Satisfaction Problem (CSP):

"A problem where a solution must satisfy a number of constraints or conditions imposed on variables."



Components of CSP

1. Variables

- Elements that need values
👉 Example: X, Y, Z
-

2. Domains

- Possible values for each variable
👉 Example: $X \in \{1, 2, 3\}$
-

3. Constraints

- Rules that restrict values
👉 Example: $X \neq Y$
-



Goal



Assign values to all variables such that:

- All constraints are satisfied ✓
-



Example



Map Coloring Problem

- Variables → Regions
 - Domain → Colors (Red, Green, Blue)
 - Constraint → Adjacent regions must have different colors
-

◆ Sudoku

- Variables → Cells
 - Domain → Numbers (1–9)
 - Constraint → No repetition in row, column, box
-

Types of Constraints

1. Unary Constraint

- Applies to single variable
👉 Example: $X > 5$
-

2. Binary Constraint

- Between two variables
👉 Example: $X \neq Y$
-

3. Global Constraint

- Involves multiple variables
👉 Example: All values must be different

2.1.13 Iterative Deepening Depth-First Search (IDDFS)



2.1.13 Iterative Deepening Depth-First Search (IDDFS)

Iterative Deepening DFS (IDDFS) is a search algorithm that combines the **advantages of Depth-First Search (DFS)** and **Breadth-First Search (BFS)**.

👉 It performs **DFS repeatedly with increasing depth limits** until the goal is found.



Definition

👉 **Iterative Deepening DFS:**

"A search algorithm that repeatedly applies depth-limited DFS with increasing depth until the goal state is reached."



Key Idea

- First search with depth limit = 0
- Then depth limit = 1
- Then depth limit = 2
- Continue until goal is found

👉 Combines:

- DFS → low memory
- BFS → completeness



Working Steps

1. Set depth limit = 0
2. Perform **DFS up to that depth**
3. If goal not found → increase depth
4. Repeat until goal is found



Example

```
Depth = 0 → A
Depth = 1 → A → B, C
Depth = 2 → A → B → D, E and A → C → F
```

👉 Continues until goal is found

Advantages

- ✓ Complete (guaranteed to find solution)
- ✓ Uses less memory (like DFS)
- ✓ Finds shortest path (like BFS)

Disadvantages

- ✗ Repeats work (nodes are explored multiple times)
- ✗ Slightly slower than BFS

Time & Space Complexity

- **Time Complexity:** $O(b^d)$
- **Space Complexity:** $O(b \times d)$

Where:

- **b** = branching factor
- **d** = depth of solution

Comparison

Feature	IDDFS	DFS	BFS
Memory	Low	Low	High
Completeness	✓ Yes	✗ No	✓ Yes
Optimal	✓ Yes	✗ No	✓ Yes

Real-Life Example

👉 Searching for a file:

- First check shallow folders
- Then go deeper step by step

When to Use IDDFS

- When depth of solution is unknown
- When memory is limited

- When shortest path is required

2.1.14 Bidirectional Search



2.1.14 Bidirectional Search (AI)

Bidirectional Search is a search technique that runs **two simultaneous searches**:

- One from the **initial state (forward search)**
- One from the **goal state (backward search)**

👉 The search stops when **both meet in the middle**.



Definition

👉 **Bidirectional Search:**

"A search algorithm that simultaneously searches forward from the start state and backward from the goal state until the two searches meet."



Key Idea

- Instead of searching the whole space from one side,
- **Divide the work into two halves**

👉 This makes it **much faster** than normal search.



Working Steps

1. Start forward search from initial node
2. Start backward search from goal node
3. Expand nodes in both directions
4. Check for intersection (common node)
5. Combine paths to get final solution



Example

```
Start → A → B → C
Goal  → X → Y → C
```

👉 Meeting point = **C**

👉 Final path = Start → A → B → C → Y → X → Goal



Complexity

- **Time Complexity:** $O(b^{(d/2)})$
- **Space Complexity:** $O(b^{(d/2)})$

👉 Much better than:

- BFS $\rightarrow O(b^d)$



Advantages

- ✓ Faster than BFS
- ✓ Reduces search space significantly
- ✓ Efficient for large problems



Disadvantages

- ✗ Requires knowing the goal state
- ✗ Hard to implement backward search
- ✗ Needs extra memory



Comparison with BFS

Feature	Bidirectional Search	BFS
Direction	Two-way	One-way
Time	Faster	Slower
Complexity	$O(b^{(d/2)})$	$O(b^d)$
Memory	High	High



Real-Life Example

- 👉 Finding shortest route:
- One search from your location
 - One from destination
 - Meet in the middle



When to Use

- When **initial and goal states are known**

- When search space is large
- When reverse operations are possible

2.2.1 AI and Game Playing



2.2.1 AI and Game Playing

Game Playing in AI is one of the most important applications of Artificial Intelligence where machines are designed to **play games intelligently** by making decisions, planning moves, and competing against humans or other machines.



Definition



AI in Game Playing:

"The use of Artificial Intelligence techniques to design systems that can play games by making optimal decisions based on the current state."



Objectives of Game Playing AI

- Make **optimal moves**
 - Predict opponent's actions
 - Maximize chances of **winning**
 - Minimize opponent's advantage
-



Key Concepts

1. Game Tree

- A tree structure representing all possible moves
 - Nodes → Game states
 - Edges → Moves
-

2. Players

- **MAX (Maximizer)**: Tries to maximize score
 - **MIN (Minimizer)**: Tries to minimize score
-

3. Utility Function

- Assigns value to game states
 - 👉 Example:
- Win = +1

- Loss = -1
 - Draw = 0
-

4. Strategy

- A plan of actions to win the game
-

Types of Games in AI

1. Deterministic Games

- No randomness
 - 👉 Examples: Chess, Tic-Tac-Toe
-

2. Stochastic Games

- Include randomness
 - 👉 Examples: Ludo, Poker
-

3. Perfect Information Games

- All information visible
 - 👉 Example: Chess
-

4. Imperfect Information Games

- Hidden information
 - 👉 Example: Poker
-

Techniques Used in Game Playing

1. Minimax Algorithm

- Simulates all possible moves
 - Chooses best move assuming opponent plays optimally
-

2. Alpha-Beta Pruning

- Optimizes minimax by cutting unnecessary branches
-

3. Heuristic Evaluation

- Estimates value of game states
-



Famous Examples

- Chess-playing AI → defeated world champions
 - Tic-Tac-Toe AI → always draws or wins
 - Video game bots
-



Advantages

- ✓ Improves decision-making
 - ✓ Useful in strategy development
 - ✓ Real-world applications beyond games
-



Disadvantages

- ✗ High computation for complex games
 - ✗ Requires strong heuristics
 - ✗ Difficult for games with uncertainty
-



Applications

- Board games (Chess, Checkers)
- Video games
- Military strategy simulations
- Business decision-making

2.2.2 Move Generator & Static Evaluation (AI in Game Playing)



2.2.2 Move Generator & Static Evaluation (AI in Game Playing)

In AI-based game playing (like Chess or Tic-Tac-Toe), two important components help the system decide the **best move**:

👉 **Move Generator**

👉 **Static Evaluation Function**

◆ 1. Move Generator

🧠 Definition

👉 **Move Generator:**

"A component that generates all possible legal moves from a given game state."

🔍 Key Idea

- Takes the **current board state**
 - Produces all **valid next moves**
 - Forms the **game tree**
-

⚙️ Working

1. Input: Current game position
 2. Apply game rules
 3. Generate all possible moves
 4. Output: List of valid next states
-

📊 Example

👉 In Tic-Tac-Toe:

- Empty cells → possible moves

👉 In Chess:

- Each piece → different valid moves
-

🎯 Importance

- Expands the **search tree**

- Provides options for decision-making
-

♦ 2. Static Evaluation Function

Definition

Static Evaluation Function:

"A function that assigns a numerical value to a game state to estimate how good or bad it is."

Key Idea

- Evaluates position **without playing till the end**
 - Used when full search is not possible
-

Working

1. Input: Game state
 2. Analyze features (position, pieces, advantage)
 3. Assign score
-

Example

In Chess:

- King safety
- Number of pieces
- Board control

Example score:

- Winning position → +10
 - Losing position → -10
-

Importance

- Helps AI choose best move
- Used in algorithms like **Minimax**

2.2.2 Move Generator & Static Evaluation (AI in Game Playing)



2.2.2 Move Generator & Static Evaluation (AI in Game Playing)

In AI-based game playing systems (like Chess or Tic-Tac-Toe), two core components help the system **decide the best move**:

👉 **Move Generator**

👉 **Static Evaluation Function**

◆ 1. Move Generator

🧠 Definition

👉 **Move Generator:**

"A component that generates all possible legal moves from a given game state."

🔍 Key Idea

- Takes the **current position of the game**
 - Applies game rules
 - Produces all **valid next moves (successor states)**
-

⚙️ Working

1. Input → Current game state
 2. Apply rules of the game
 3. Generate all legal moves
 4. Output → List of possible next states
-

📊 Example

🎮 Tic-Tac-Toe

- If 3 empty cells → 3 possible moves

♟️ Chess

- Each piece has specific movement rules
 - Move generator lists all legal moves for current player
-

🎯 Importance

- ✓ Builds the **game tree**
 - ✓ Provides choices for AI
 - ✓ First step in decision-making
-

◆ 2. Static Evaluation Function

🧠 Definition

👉 Static Evaluation Function:

"A function that assigns a numerical value to a game state to estimate its desirability."

🔍 Key Idea

- Evaluates a position **without exploring the full game tree**
 - Used when searching all possibilities is not feasible
-

⚙️ Working

1. Input → Current game state
 2. Analyze important features
 3. Assign a numerical score
-

📊 Example

♟️ Chess Evaluation Factors

- Material (number of pieces)
- King safety
- Board control

👉 Example scoring:

- Strong position → +10
 - Weak position → -10
-

🎯 Importance

- ✓ Helps AI choose best move
- ✓ Guides algorithms like **Minimax**

✓ Reduces computation time

Move Generator vs Static Evaluation

Feature	Move Generator	Static Evaluation Function
Purpose	Generate moves	Evaluate states
Output	List of moves	Numerical score
Role	Expands search tree	Guides decision-making

How They Work Together

1. **Move Generator** → creates all possible moves
2. **Search Algorithm (Minimax)** → explores moves
3. **Static Evaluation** → scores positions
4. Best move is selected

Real-Life Analogy

👉 Playing Chess:

- You think of all possible moves (**move generator**)
- Then judge which move is best (**evaluation**)

2.2.3 Game Playing Strategies (AI)



2.2.3 Game Playing Strategies (AI)

In Artificial Intelligence, **Game Playing Strategies** are techniques used by AI systems to **decide the best move** in a game by analyzing possible future states and opponent actions.



Definition



Game Playing Strategies:

"Methods used by AI to select optimal moves in a game by evaluating possible actions and predicting outcomes."



Objectives

- Maximize **chance of winning**
- Minimize opponent's advantage
- Make **optimal decisions**
- Predict opponent's moves



Types of Game Playing Strategies

◆ 1. Minimax Strategy

- Used in **two-player games**
- One player tries to **maximize (MAX)** score
- Other tries to **minimize (MIN)** score



AI assumes opponent plays optimally



Finds best possible move

◆ 2. Alpha-Beta Pruning

- Optimization of **Minimax**
- Removes unnecessary branches



Reduces computation time



Faster than Minimax

◆ 3. Heuristic Strategy

- Uses **evaluation functions**
- Estimates best move without exploring full tree

✓ Useful for complex games

◆ 4. Depth-Limited Search

- Limits search to a fixed depth
- Uses evaluation function at cutoff

✓ Saves time and memory

◆ 5. Iterative Deepening

- Repeats depth-limited search with increasing depth

✓ Combines efficiency + accuracy

◆ 6. Expectiminimax Strategy

- Used in **games with randomness**
👉 Example: Ludo, Dice games

✓ Handles uncertainty

⚙️ Key Concepts Used

- Game Tree
 - Utility Function
 - Move Generator
 - Static Evaluation Function
-

⚖️ Advantages

- ✓ Helps AI make intelligent decisions
 - ✓ Works in competitive environments
 - ✓ Applicable in real-world planning
-

✗ Disadvantages

- ✗ High computational cost
- ✗ Requires good evaluation function
- ✗ Complex for large games

Applications

- Chess AI
- Video games
- Robotics decision-making
- Business strategy simulations

Real-Life Example

 Playing Chess:

- Think of all moves
- Predict opponent's response
- Choose best move

2.2.4 Problems in Game Playing (AI)



2.2.4 Problems in Game Playing (AI)

In Artificial Intelligence, **game playing systems** face several challenges while making optimal decisions due to the **complexity of games and uncertainty of opponent behavior**.



Definition



Problems in Game Playing:

"Difficulties faced by AI systems while analyzing game states and selecting the best move efficiently."



Major Problems in Game Playing

◆ 1. Large Search Space

- Games like Chess have **millions of possible moves**
- Exploring all possibilities is impractical



Leads to **high time complexity**

◆ 2. Time Constraints

- AI must make decisions within limited time
- Cannot explore full game tree



Needs faster strategies

◆ 3. Optimal Move Selection

- Difficult to always choose the **best move**
 - Depends on evaluation function
-

◆ 4. Uncertainty of Opponent

- Opponent may not play optimally
 - Hard to predict opponent's strategy
-

◆ 5. Imperfect Information

- Some games hide information
 - 👉 Example: Card games
-

◆ 6. Evaluation Function Limitations

- Static evaluation may not reflect true game state
- Poor evaluation → wrong decisions

◆ 7. Depth Limitation

- Cannot search indefinitely
- Must limit search depth

👉 May miss better moves deeper in tree

◆ 8. Computational Cost

- High memory and processing requirements
- Especially for complex games

⚙️ Solutions / Techniques

To overcome these problems, AI uses:

- **Heuristic Search** → Reduces search space
- **Alpha-Beta Pruning** → Cuts unnecessary nodes
- **Iterative Deepening** → Balances time and depth
- **Evaluation Functions** → Estimate best moves

⚖️ Summary Table

Problem	Description	Solution Technique
Large search space	Too many possibilities	Heuristics
Time constraint	Limited decision time	Depth-limited search
Poor evaluation	Wrong scoring	Better evaluation function
Uncertainty	Opponent unpredictability	Probabilistic models

💡 Real-Life Example

👉 Chess AI:

- Cannot check all moves

- Uses heuristics + pruning to decide quickly
-

Importance

- Helps improve AI game performance
- Leads to development of better algorithms
- Useful in real-world decision-making

Module III: Logic and Knowledge Representation

3.1.1 Knowledge Representation (AI)



3.1.1 Knowledge Representation (AI)

Knowledge Representation (KR) in Artificial Intelligence is the way we **store, organize, and structure knowledge** so that a computer system can understand it and use it to solve problems.

◆ What is Knowledge Representation?

It is a method of representing real-world information in a form that a machine can:

- Understand
- Reason about
- Use to make decisions

👉 In simple terms:

KR = Converting human knowledge → machine-understandable format

◆ Why is Knowledge Representation Important?

- Helps in **decision making**
 - Enables **reasoning & inference**
 - Used in **expert systems**, chatbots, recommendation systems
 - Improves **problem-solving capability of AI systems**
-

◆ Types of Knowledge in AI

1. Declarative Knowledge

- Facts about the world
- Example: "Delhi is the capital of India"

2. Procedural Knowledge

- How to perform tasks
- Example: Steps to solve a problem

3. Heuristic Knowledge

- Experience-based knowledge (rules of thumb)

4. Meta Knowledge

- Knowledge about other knowledge

◆ Techniques of Knowledge Representation

1. Logical Representation

- Uses logic (like propositional & predicate logic)
- Example:

```
Human(Socrates)
Human(x) → Mortal(x)
⇒ Mortal(Socrates)
```

2. Semantic Networks

- Graph structure with:
 - Nodes → Objects/Concepts
 - Edges → Relationships

Example:

```
Dog → is a → Animal
Animal → has → Life
```

3. Frame Representation

- Uses data structures (like objects in programming)
- Contains:
 - Slots (attributes)
 - Values

Example:

```
Frame: Car
Color: Red
Wheels: 4
```

4. Production Rules

- IF-THEN rules

Example:

```
IF temperature > 100  
THEN fever = true
```

♦ Characteristics of Good Knowledge Representation

- **Accuracy** – Correct representation
- **Completeness** – Covers all required info
- **Efficiency** – Fast processing
- **Flexibility** – Easy to update

♦ Real-Life Example

Think of:

- **Google Maps** → uses KR to represent roads & routes
- **ChatGPT** → uses KR to understand and respond
- **Medical Expert Systems** → use rules & logic

♦ Summary

Knowledge Representation is the **foundation of AI systems**, enabling machines to:

- Think
- Learn
- Make decisions

3.1.2 Structured Knowledge (AI)



3.1.2 Structured Knowledge (AI)

Structured Knowledge in Artificial Intelligence refers to knowledge that is **organized in a well-defined, systematic format**, making it easy for machines to process, store, and retrieve.

♦ What is Structured Knowledge?

It is knowledge arranged in a **clear structure (like tables, objects, or hierarchies)** where relationships between data are explicitly defined.

👉 In simple terms:

Structured Knowledge = Organized + Machine-readable data

♦ Key Features

- Clearly defined **format/schema**
- Easy to **search and query**
- Supports **relationships between entities**
- Efficient for **reasoning and inference**

♦ Examples of Structured Knowledge

1. Database Tables

Student_ID	Name	Course
101	Ravi	BCA

✓ Data is organized in rows & columns

✓ Easy to query using SQL

2. Knowledge Graphs

- Represent knowledge using nodes and relationships

Example:

```
Ravi → studies → BCA
BCA → belongs to → Computer Science
```

3. Object-Based Structures (Frames)

Similar to objects in programming:

```
Student:  
  Name: Ravi  
  Age: 20  
  Course: BCA
```

◆ Types of Structured Knowledge Representation

1. Relational Model

- Data stored in tables
- Used in databases like MySQL

2. Hierarchical Model

- Tree-like structure
- Parent → Child relationship

3. Network Model

- Graph-based structure
- More flexible than hierarchy

3.1.3 Associative Networks (AI)



3.1.3 Associative Networks (AI)

Associative Networks in Artificial Intelligence are a way of representing knowledge using a **network of interconnected nodes**, where each node represents a concept and links represent relationships or associations.

◆ What is an Associative Network?

It is a **graph-based structure** where:

- **Nodes** → represent objects, concepts, or ideas
- **Links (edges)** → represent relationships between them

👉 In simple terms:

Associative Network = Concepts connected like a web

◆ Basic Structure

Example:

```
Dog → is a → Animal  
Dog → has → Tail  
Animal → needs → Food
```

Here:

- Dog, Animal = Nodes
- "is a", "has", "needs" = Relationships

◆ Key Characteristics

- Represents knowledge in a **linked form**
- Supports **inheritance of properties**
- Easy to visualize relationships
- Works similar to **human memory associations**

◆ Types of Relationships

1. IS-A Relationship

- Shows class/subclass

- Example: Dog → is a → Animal

2. HAS-A Relationship

- Shows attributes
- Example: Dog → has → Tail

3. CAUSE-EFFECT / ASSOCIATION

- Example: Rain → causes → Wet roads

◆ Important Concept: Inheritance

Inheritance means:

👉 A child node inherits properties from parent node

Example:

```
Bird → has → Wings  
Sparrow → is a → Bird
```

✓ So, Sparrow also has wings (inherited property)

◆ Advantages

- Simple and intuitive
- Easy to represent relationships
- Supports inheritance
- Good for semantic understanding

◆ Disadvantages

- Can become complex for large data
- Not efficient for detailed logical reasoning
- Ambiguity in relationships

◆ Real-Life Applications

- **Knowledge Graphs (Google Search)**
- **Recommendation Systems**

- **Natural Language Processing (NLP)**
- **Expert Systems**

3.1.4 Frame Structures (AI)



3.1.4 Frame Structures (AI)

Frame Structures in Artificial Intelligence are a knowledge representation technique where information is stored in **structured units called frames**, similar to **objects in programming**.

♦ What is a Frame?

A **frame** is a data structure used to represent a concept, object, or situation using:

- **Slots** → Attributes (properties)
- **Values** → Data stored in those slots

👉 In simple terms:

Frame = Object with properties + values

♦ Basic Structure of a Frame

Example:

```
Frame: Student
-----
Name: Ravi
Age: 20
Course: BCA
```

Here:

- Frame → Student
- Slots → Name, Age, Course
- Values → Ravi, 20, BCA

♦ Types of Frames

1. Simple Frame

- Represents a single object

Example:

```
Frame: Car  
Color: Red  
Wheels: 4
```

2. Hierarchical Frames

- Frames can inherit properties from other frames

Example:

```
Frame: Vehicle  
Wheels: 4  
  
Frame: Car (inherits Vehicle)  
Color: Red
```

✓ Car inherits "Wheels = 4" from Vehicle

◆ Key Concepts

1. Slots

- Attributes of a frame
- Example: Color, Age

2. Facets

- Additional information about slots
- Example: default value, range

3. Inheritance

- Child frames inherit properties from parent frames

◆ Characteristics

- Structured and organized
 - Similar to **object-oriented programming (OOP)**
 - Supports inheritance
 - Easy to update and modify
-

◆ **Advantages**

- Easy to understand and implement
 - Supports reusability (via inheritance)
 - Efficient for representing structured data
 - Natural representation of real-world objects
-

◆ **Disadvantages**

- Not suitable for complex logical reasoning
 - Can become large and difficult to manage
 - Limited flexibility compared to other methods
-

◆ **Real-Life Applications**

- **Expert Systems**
- **Natural Language Processing (NLP)**
- **Robotics (object representation)**
- **Game AI**

3.1.5 Conceptual Dependencies (AI)



3.1.5 Conceptual Dependencies (AI)

Conceptual Dependencies (CD) in Artificial Intelligence is a knowledge representation technique used to represent the **meaning of natural language sentences** in a structured and language-independent form.

◆ What is Conceptual Dependency?

It is a method that focuses on:

👉 **"What a sentence means" rather than "how it is written"**

- Removes ambiguity in language
- Represents actions using **standard conceptual primitives**

👉 In simple terms:

CD = Representing meaning of sentences in a universal format

◆ Why Conceptual Dependency?

Natural language can be:

- Ambiguous
- Complex
- Different for same meaning

Example:

- "Ram gave a book to Shyam"
- "Shyam received a book from Ram"

✓ Both mean the same → CD represents them in **one unified form**

◆ Basic Idea of CD

- Uses **primitive actions (ACTs)**
 - Defines relationships between:
 - Actor
 - Object
 - Action
-

◆ Common Primitive Actions (ACTs)

Action	Meaning
ATRANS	Transfer of ownership
PTRANS	Physical movement
MTRANS	Transfer of information
PROPEL	Apply physical force
INGEST	Eating/consuming
EXPEL	Removing something

◆ Example Representation

Sentence:

👉 "Ram gave a book to Shyam"

Conceptual Dependency Form:

```
ATRANS
Actor: Ram
Object: Book
From: Ram
To: Shyam
```

✓ Focus is on **action (transfer)** and participants

◆ Another Example

Sentence:

👉 "She went to school"

```
PTRANS
Actor: She
From: Home
To: School
```

◆ Key Characteristics

- Language-independent representation
- Reduces ambiguity

- Focuses on **semantics (meaning)**
 - Uses predefined action primitives
-

◆ **Advantages**

- Same meaning → same representation
 - Useful in **Natural Language Processing (NLP)**
 - Helps in **machine understanding of text**
-

◆ **Disadvantages**

- Limited set of primitive actions
 - Complex to implement
 - Not suitable for all types of sentences
-

◆ **Applications**

- **Chatbots & Virtual Assistants**
- **Machine Translation**
- **Story Understanding Systems**
- **Question Answering Systems**

3.1.6 Scripts (AI)



3.1.6 Scripts (AI)

Scripts in Artificial Intelligence are a knowledge representation technique used to describe **a sequence of events in a particular context or situation**.

◆ What is a Script?

A **script** represents **stereotypical (common) situations** that happen in everyday life, in a fixed order.

👉 In simple terms:

Script = Predefined sequence of events

◆ Why Scripts are Used?

- To understand **real-world situations**
 - To fill in **missing information automatically**
 - To make AI systems behave like humans in understanding events
-

◆ Basic Components of a Script

1. Entry Conditions

- Conditions required before the script starts
 - Example: You are hungry (for restaurant script)
-

2. Roles

- People or objects involved
 - Example: Customer, Waiter, Chef
-

3. Props

- Objects used in the script
 - Example: Menu, Table, Food
-

4. Scenes

- Sequence of events (main part)

Example:

1. Enter restaurant

2. Order food
3. Eat food
4. Pay bill
5. Leave

5. Results

- Outcome of the script
- Example: Hunger satisfied

◆ Example: Restaurant Script

Entry Condition: Hungry person

Roles:

- Customer
- Waiter

Props:

- Table
- Menu
- Food

Scenes:

1. Customer enters restaurant
2. Customer orders food
3. Waiter serves food
4. Customer eats
5. Customer pays bill

Result:

- Customer is no longer hungry

◆ Key Characteristics

- Represents **temporal sequence (time order)**
- Based on **real-life experiences**
- Helps in **predicting next events**
- Reduces need for explicit information

◆ Advantages

- Easy to understand real-world scenarios
 - Helps AI infer missing steps
 - Useful in **story understanding & NLP**
-

◆ **Disadvantages**

- Limited to **specific situations**
 - Not flexible for unexpected events
 - Requires predefined scripts
-

◆ **Applications**

- **Chatbots & Virtual Assistants**
- **Story Understanding Systems**
- **Game AI**
- **Robotics (task execution)**

3.2.1 Propositional Logic (AI)



3.2.1 Propositional Logic (AI)

Propositional Logic in Artificial Intelligence is a formal way of representing knowledge using **statements (propositions)** that are either **true or false**.

◆ What is Propositional Logic?

A **proposition** is a declarative statement that has:

- **Truth value = True (T) or False (F)**

👉 In simple terms:

Propositional Logic = Logic using true/false statements

◆ Examples of Propositions

- "It is raining" → True/False
- "2 + 2 = 4" → True
- "Delhi is in USA" → False

✓ These are valid propositions because they can be evaluated

✗ Not propositions:

- "Close the door" (command)
- "What is your name?" (question)

◆ Logical Connectives

These are used to combine propositions:

Symbol	Name	Meaning	Example
$\neg P$	NOT	Negation	NOT raining
$P \wedge Q$	AND	Both true	Rain AND cold
$P \vee Q$	OR	At least one true	Rain OR cold
$P \rightarrow Q$	Implication	If P then Q	If rain → wet
$P \leftrightarrow Q$	Biconditional	Both same	P iff Q

◆ Truth Table Example (AND)

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

◆ Representation Example

Statement:

👉 "If it rains, then the ground is wet"

Let:

- P = It rains
- Q = Ground is wet

Representation:

$P \rightarrow Q$

◆ Important Concepts

1. Well-Formed Formula (WFF)

- A valid logical expression
- Example: $(P \wedge Q)$, $(P \rightarrow Q)$

2. Tautology

- Always true
- Example: $P \vee \neg P$

3. Contradiction

- Always false
- Example: $P \wedge \neg P$

4. Contingency

- Sometimes true, sometimes false

◆ Advantages

- Simple and easy to understand
 - Clear semantics (true/false)
 - Useful in **basic reasoning systems**
-

◆ Disadvantages

- Cannot represent complex relationships
 - No support for variables or quantifiers
 - Limited expressiveness
-

◆ Applications

- **Digital Circuits Design**
 - **Rule-Based Systems**
 - **AI Reasoning Systems**
 - **Software Verification**
-

◆ Summary

Propositional Logic:

- Uses **true/false statements**
- Combines them using **logical operators**
- Forms the **foundation of logical reasoning in AI**

3.2.2 First Order Predicate Logic (FOPL)



3.2.2 First Order Predicate Logic (FOPL)

First Order Predicate Logic (FOPL) in Artificial Intelligence is an advanced form of logic that represents knowledge using **objects, predicates, and quantifiers**, making it much more powerful than propositional logic.

♦ What is FOPL?

FOPL allows us to describe:

- **Objects (things)**
- **Properties of objects**
- **Relationships between objects**

👉 In simple terms:

FOPL = Propositional Logic + Variables + Relationships

♦ Basic Components

1. Constants

- Specific objects
 - Example: Ram, Delhi
-

2. Variables

- Represent general objects
 - Example: x , y
-

3. Predicates

- Describe properties or relationships

Examples:

- $\text{Human}(x) \rightarrow x \text{ is human}$
 - $\text{Loves}(\text{Ram}, \text{Sita}) \rightarrow \text{Ram loves Sita}$
-

4. Functions

- Map objects to another object

Example:

- $\text{Father}(\text{Ram}) \rightarrow$ returns Ram's father

5. Quantifiers

a) Universal Quantifier ($\forall$)

- Means "for all"

Example:

👉 All humans are mortal

```
 $\forall x \text{ (Human}(x) \rightarrow \text{Mortal}(x))$ 
```

b) Existential Quantifier ($\exists$)

- Means "there exists"

Example:

👉 Someone is a doctor

```
 $\exists x \text{ (Doctor}(x))$ 
```

◆ Example Representation

Statement:

👉 "All humans are mortal"

Instead of writing separate statements like in propositional logic:

- $\text{Human}(\text{Ram}) \rightarrow \text{Mortal}(\text{Ram})$
- $\text{Human}(\text{Shyam}) \rightarrow \text{Mortal}(\text{Shyam})$

✓ FOPL simplifies it:

```
 $\forall x \text{ (Human}(x) \rightarrow \text{Mortal}(x))$ 
```

Another Example:

👉 "Ram loves Sita"

Loves(Ram, Sita)

◆ Key Features

- Uses **variables and quantifiers**
- Can represent **complex relationships**
- More expressive than propositional logic

◆ FOPL vs Propositional Logic

Feature	Propositional Logic	FOPL
Variables	✗ No	✓ Yes
Relationships	✗ No	✓ Yes
Expressiveness	Low	High
Example	$P \rightarrow Q$	$\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

◆ Advantages

- Highly expressive
- Represents real-world knowledge better
- Supports logical reasoning

◆ Disadvantages

- More complex
- Requires more computation
- Harder to implement

◆ Applications

- **Expert Systems**
- **Natural Language Processing (NLP)**
- **Database Query Systems**
- **AI Reasoning Engines**

◆ Summary

First Order Predicate Logic:

- Extends propositional logic
- Uses **predicates, variables, and quantifiers**
- Enables AI to represent **complex real-world knowledge**

3.2.3 Conversion to Clausal Form (AI)



3.2.3 Conversion to Clausal Form (AI)

Conversion to Clausal Form in Artificial Intelligence is the process of transforming logical expressions (especially in **First Order Predicate Logic**) into a **standard format called clauses**, which is required for applying inference techniques like **resolution**.

◆ What is Clausal Form?

- A **clause** is a disjunction (OR) of literals
- Example:

$$(P \vee \neg Q \vee R)$$

👉 In simple terms:

Clausal Form = Set of clauses used for logical reasoning

◆ Why Convert to Clausal Form?

- Required for **Resolution Algorithm**
- Makes reasoning **systematic and efficient**
- Standard format for **automated theorem proving**

◆ Steps for Conversion to Clausal Form

Let's understand step-by-step 👉

✅ Step 1: Eliminate Implication ($\rightarrow$)

Replace:

$$P \rightarrow Q \equiv \neg P \vee Q$$

✅ Step 2: Move NOT ($\neg$) inward

Use rules:

- $\neg(P \wedge Q) \rightarrow \neg P \vee \neg Q$

- $\neg(P \vee Q) \rightarrow \neg P \wedge \neg Q$

✓ Step 3: Standardize Variables

Rename variables so that each quantifier has **unique variables**

✓ Step 4: Move Quantifiers to Front

Convert to **Prenex Form**

(All quantifiers at beginning)

✓ Step 5: Eliminate Existential Quantifier ($\exists$)

Use **Skolemization**:

- Replace $\exists$ with a constant or function

Example:

$$\exists x P(x) \rightarrow P(a)$$

✓ Step 6: Drop Universal Quantifier ($\forall$)

- Assume all variables are universally quantified

✓ Step 7: Convert to CNF (Conjunctive Normal Form)

- CNF = AND of ORs

Example:

$$(P \vee Q) \wedge (R \vee S)$$

✓ Step 8: Split into Clauses

Each part becomes a separate clause

Example:

$$(P \vee Q), (R \vee S)$$

◆ Example Conversion

Statement:

👉 $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

Step-by-step:

1. Remove implication:

```

$$\forall x (\neg \text{Human}(x) \vee \text{Mortal}(x))$$

```

1. Drop $\forall$:

```

$$\neg \text{Human}(x) \vee \text{Mortal}(x)$$

```

Final Clause:

```

$$(\neg \text{Human}(x) \vee \text{Mortal}(x))$$

```

◆ Another Example

Statement:

👉 $\exists x (\text{Dog}(x) \wedge \text{Loves}(x, \text{Bone}))$

After Skolemization:

```

$$\text{Dog}(a) \wedge \text{Loves}(a, \text{Bone})$$

```

Clauses:

```

$$\text{Dog}(a), \text{Loves}(a, \text{Bone})$$

```

◆ Key Concepts

- **Literal** → Predicate or its negation
- **Clause** → OR of literals

- **CNF** → AND of clauses
-

♦ **Advantages**

- Standard format for reasoning
 - Easy to apply **resolution**
 - Used in **automated theorem proving**
-

♦ **Disadvantages**

- Conversion process is lengthy
 - Can increase complexity of expressions
-

♦ **Applications**

- **AI reasoning systems**
 - **Logic programming (Prolog)**
 - **Automated theorem proving**
-

♦ **Summary**

Conversion to Clausal Form:

- Converts logical expressions into **standard clause format**
- Essential for **resolution-based reasoning**
- Plays a key role in **AI problem solving**

3.2.4 Inference Rules (AI)



3.2.4 Inference Rules (AI)

Inference Rules in Artificial Intelligence are logical rules used to **derive new conclusions (facts) from existing knowledge**.

◆ What are Inference Rules?

They are **valid logical patterns** that help in reasoning.

👉 In simple terms:

Inference Rules = Methods to derive new knowledge from known facts

◆ Why are Inference Rules Important?

- Enable **logical reasoning**
- Help AI systems **make decisions**
- Used in **expert systems & theorem proving**
- Foundation of **automated reasoning**

◆ Common Inference Rules

1. Modus Ponens (MP) 🔥 (Most Important)

If:

- $P \rightarrow Q$
- P is true

Then:

- Q is true

Example:

```
If it rains → ground is wet
It rains
⇒ Ground is wet
```

2. Modus Tollens (MT)

If:

- $P \rightarrow Q$
- Q is false

Then:

- P is false

Example:

```
If it rains  $\rightarrow$  ground is wet  
Ground is not wet  
 $\Rightarrow$  It did not rain
```

3. Hypothetical Syllogism

If:

- $P \rightarrow Q$
- $Q \rightarrow R$

Then:

- $P \rightarrow R$

Example:

```
If I study  $\rightarrow$  I pass  
If I pass  $\rightarrow$  I get job  
 $\Rightarrow$  If I study  $\rightarrow$  I get job
```

4. Disjunctive Syllogism

If:

- $P \vee Q$
- $\neg P$

Then:

- Q

Example:

```
I will go to Delhi OR Mumbai  
I will not go to Delhi  
⇒ I will go to Mumbai
```

5. Addition Rule

If:

- P is true

Then:

- $P \vee Q$ is true

6. Simplification Rule

If:

- $P \wedge Q$

Then:

- P (or Q)

7. Resolution Rule 🔥 (Very Important)

If:

- $(P \vee Q)$
- $(\neg Q \vee R)$

Then:

- $(P \vee R)$

👉 Used in **automated theorem proving**

◆ Inference in Predicate Logic (FOPL)

1. Universal Instantiation (UI)

From:

```
∀x P(x)
```

We can derive:

$P(a)$

2. Existential Instantiation (EI)

From:

$\exists x P(x)$

We can derive:

$P(a)$

◆ Key Concepts

- **Premises** → Given facts
- **Conclusion** → Derived fact
- **Validity** → Logical correctness

◆ Advantages

- Systematic reasoning
- Helps in building intelligent systems
- Used in logic programming

◆ Applications

- **Expert Systems**
- **AI reasoning engines**
- **Mathematical proofs**
- **Chatbots & NLP systems**

◆ Summary

Inference Rules:

- Provide **logical steps to derive conclusions**
- Core part of **AI reasoning**
- Essential for **problem solving and decision making**

3.2.5 Unification (AI)



3.2.5 Unification (AI)

Unification in Artificial Intelligence is a process used in **First Order Predicate Logic (FOPL)** to make two logical expressions **identical** by finding suitable substitutions.

♦ What is Unification?

Unification tries to answer:

👉 **"Can we make two expressions look the same?"**

If yes → find substitutions

If no → unification fails

👉 In simple terms:

Unification = Matching expressions by replacing variables

♦ Why Unification is Important?

- Core part of **Resolution Algorithm**
- Used in **automated reasoning**
- Essential in **logic programming (Prolog)**

♦ Basic Idea

We replace variables with:

- Constants
- Other variables
- Functions

so that both expressions become identical

♦ Examples of Unification

✅ Example 1:

$P(x)$ and $P(a)$

✓ Substitution:

```
x = a
```

✓ Result: Both become **P(a)**

✓ Example 2:

```
Loves(x, y) and Loves(Ram, Sita)
```

✓ Substitution:

```
x = Ram  
y = Sita
```

✗ Example 3 (Failure):

```
P(x) and Q(x)
```

✗ Cannot unify (different predicates)

◆ Most General Unifier (MGU)

- The **simplest substitution** that makes expressions identical
- Avoids unnecessary complexity

Example:

```
P(x, a) and P(b, y)
```

✓ MGU:

```
x = b  
y = a
```

◆ Unification Algorithm (Steps)

1. Compare both expressions
 2. Match predicates and arguments
 3. Replace variables where needed
 4. Repeat until expressions match
 5. If conflict occurs → fail
-

◆ **Key Rules**

- Constants must match exactly
 - Functions must match structurally
 - Variables can be substituted
 - No circular substitution (e.g., $x = f(x)$)
-

◆ **Applications**

- **Resolution in AI**
 - **Prolog Programming**
 - **Natural Language Processing (NLP)**
 - **Automated Theorem Proving**
-

◆ **Advantages**

- Simplifies logical expressions
 - Enables automated reasoning
 - Efficient matching mechanism
-

◆ **Disadvantages**

- Can be complex for large expressions
 - May fail if structures differ
-

◆ **Summary**

Unification:

- Makes logical expressions **identical**

- Uses **substitution of variables**
- Key component of **resolution and inference**

3.2.6 Resolution Principle (AI) 🔥



3.2.6 Resolution Principle (AI) 🔥

The **Resolution Principle** in Artificial Intelligence is a powerful **inference rule** used to prove whether a statement is true or false by **deriving a contradiction**.

◆ What is Resolution Principle?

It is a rule that:

- 👉 Combines two clauses to produce a new clause
- 👉 Eliminates complementary literals (P and $\neg P$)
- 👉 In simple terms:

Resolution = Proving something by contradiction

◆ Basic Rule of Resolution

If we have:

```
(P ∨ Q)
(¬Q ∨ R)
```

👉 Then we can derive:

```
(P ∨ R)
```

✓ Here:

- Q and $\neg Q$ cancel each other
- Remaining literals form a new clause

◆ Why Resolution is Important?

- Used in **automated theorem proving**
- Core of **AI reasoning systems**
- Works with **clausal form (CNF)**
- Complete and sound method

◆ Steps in Resolution Method

1. **Convert all statements into Clausal Form (CNF)**
 2. **Negate the goal** (what you want to prove)
 3. Add it to the knowledge base
 4. Apply resolution rule repeatedly
 5. If you derive **empty clause** ($\square$) $\rightarrow$ PROVED
 6. If not $\rightarrow$ cannot prove
-

◆ Example (Step-by-Step)

Given:

1. $P \rightarrow Q$
2. $Q \rightarrow R$
3. P

Prove: R

Step 1: Convert to CNF

```
¬P ∨ Q
¬Q ∨ R
P
```

Step 2: Resolve

From:

```
(¬P ∨ Q) and (P)
```

👉 Result:

```
Q
```

From:

$(\neg Q \vee R)$ and (Q)

👉 Result:

R

✓ Hence proved ✓

◆ Refutation (Contradiction Method)

To prove R:

- Add $\neg R$
- Try to derive contradiction

If we get:

□ (empty clause)

✓ It means original statement is TRUE

◆ Key Concepts

- **Clause** → OR of literals
- **Literal** → Predicate or its negation
- **Empty Clause (□)** → Contradiction

◆ Advantages

- Simple and uniform rule
- Works for both propositional & predicate logic
- Complete method (can prove anything logically true)

◆ Disadvantages

- Can be computationally expensive
- Requires conversion to CNF

◆ Applications

- Automated Theorem Proving
- Logic Programming (Prolog)
- AI Reasoning Systems
- Expert Systems

3.2.7 Naive Bayes Algorithm (AI)



3.2.7 Naive Bayes Algorithm (AI)

The **Naive Bayes Algorithm** in Artificial Intelligence is a **supervised machine learning algorithm** used for **classification**, based on Bayes' Theorem.

◆ What is Naive Bayes?

- **Bayes** → uses probability theorem
- **Naive** → assumes all features are **independent**

👉 In simple terms:

It predicts the class with highest probability

◆ Formula (Core Concept)

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)}$$

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)}$$

$$P(A)P(A)P(A)$$

$$P(B | A)P(B | A)P(B | A)$$

$$P(B | \neg A)P(B | \neg A)P(B | \neg A)$$

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)} \approx 0.68, P(B) \approx 0.25$$

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)} \approx 0.68, P(B) \approx 0.25$$

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)} \approx 0.68, P(B) \approx 0.25$$

$$P(B) = 0.25, P(A | B) = 0.17, P(A | B) \approx 0.68$$

Posterior = useful evidence / total evidence

Where:

- **P(C|X)** → Probability of class given features
- **P(X|C)** → Likelihood
- **P(C)** → Prior probability
- **P(X)** → Evidence

◆ How It Works (Step-by-Step)

1. Calculate Prior Probability

- Example:
P(Spam), P(Not Spam)

2. Calculate Likelihood

- Probability of features given class
- Example:
 $P(\text{word} = \text{"free"} \mid \text{Spam})$

3. Apply Bayes Formula

- Compute probability for each class

4. Choose Maximum Probability

- Class with highest probability is selected

◆ Example (Exam Style)

Problem:

Classify email as Spam or Not Spam

Features:

- "Free" present
- "Offer" present

We compute:

```
P(Spam | Features)
P(Not Spam | Features)
```

✓ If:

- $P(\text{Spam} \mid \text{Features}) > P(\text{Not Spam} \mid \text{Features})$
👉 Email = Spam

◆ Types of Naive Bayes

1. Gaussian Naive Bayes

- For continuous data

2. Multinomial Naive Bayes 🔥

- Used in text classification

3. Bernoulli Naive Bayes

- Binary features (yes/no)

◆ Important Concept: Independence Assumption

- Assumes all features are **independent**
 - Example:
 - Words in a sentence are treated independently
-

◆ Advantages

- Fast and efficient
 - Works well with **high-dimensional data**
 - Requires less training data
 - Very effective for **NLP tasks**
-

◆ Disadvantages

- Unrealistic independence assumption
 - Cannot capture feature relationships
 - Zero probability problem (use Laplace smoothing)
-

◆ Applications

- **Spam Filtering (Email)**
- **Sentiment Analysis**
- **Document Classification**
- **Medical Diagnosis**

Module IV: Knowledge Acquisition and Expert System

4.1 Knowledge Acquisition

- 4.1.1 Types of Learning

Types of Learning



In **Artificial Intelligence (AI)**, **Types of Learning** refers to the different ways a system can acquire knowledge or improve performance based on experience or data.

Here's a **clear and structured explanation** of **4.1.1 Types of Learning** 📌

◆ 1. Supervised Learning

- Learning with **labeled data**
- Input + Correct Output (Target) is given
- Model learns to map input → output

✓ Examples:

- Email spam detection
- House price prediction

🧠 Key Idea:

Learn from **teacher (labeled dataset)**

◆ 2. Unsupervised Learning

- Learning with **unlabeled data**
- No correct output is provided
- Model finds **hidden patterns**

✓ Examples:

- Customer segmentation
- Clustering similar products

🧠 Key Idea:

Learn patterns **without guidance**

◆ 3. Semi-Supervised Learning

- Combination of:
 - Small amount of **labeled data**
 - Large amount of **unlabeled data**

✓ Examples:

- Image classification with limited labeled images

🧠 Key Idea:

Use **few labels + lots of raw data**

◆ 4. Reinforcement Learning

- Learning by **trial and error**
- Agent interacts with environment
- Receives:
 - Reward (good action)
 - Penalty (bad action)

✓ Examples:

- Game playing (Chess, Ludo AI)
- Self-driving cars

🧠 Key Idea:

Learn by **experience and rewards**

◆ 5. Rote Learning

- Memorization of facts or patterns
- No understanding or generalization

✓ Example:

- Remembering multiplication tables

🧠 Key Idea:

Store and recall only

◆ 6. Learning by Analogy

- Solve new problems using **similar past problems**

✓ Example:

- Solving a new math problem using a known method

Key Idea:

Compare and reuse knowledge

◆ 7. Explanation-Based Learning (EBL)

- Learn by **understanding explanations**
- Uses prior knowledge to generalize

✓ Example:

- Learning rules from examples + logic

Key Idea:

Learn from reasoning

◆ 8. Discovery Learning

- System discovers **rules or concepts on its own**

✓ Example:

- Finding patterns in data without help

Key Idea:

Self-exploration

Quick Summary Table

Type	Data	Guidance	Example
Supervised	Labeled	Yes	Spam detection
Unsupervised	Unlabeled	No	Clustering
Semi-Supervised	Mixed	Partial	Image classification
Reinforcement	Environment	Reward/Penalty	Games
Rote	Stored data	No	Memorization
Analogy	Past cases	Partial	Problem solving
EBL	Logic-based	Yes	Rule learning
Discovery	Raw data	No	Pattern finding

What is Knowledge Acquisition?



◆ What is Knowledge Acquisition?

Knowledge Acquisition is the process of **collecting, organizing, and storing knowledge** from different sources so that an AI system can use it to solve problems or make decisions.

👉 In simple words:

It means **“feeding knowledge into an AI system”**

◆ Sources of Knowledge

1. Human Experts

- Domain experts (doctors, engineers, teachers)
- Provide rules, experience, and insights

2. Books & Documents

- Manuals, research papers, reports

3. Databases

- Structured data (tables, records)

4. Sensors & Environment

- Real-time data (IoT devices, cameras)
-

◆ Methods of Knowledge Acquisition

1. Manual Acquisition

- Knowledge is entered **by humans**
- Example: Writing rules in an expert system

👉 Slow but accurate

2. Automatic Acquisition

- System learns knowledge **automatically from data**
- Uses **Machine Learning**

👉 Fast but depends on data quality

3. Semi-Automatic Acquisition

- Combination of:
 - Human input
 - Machine learning

👉 Most practical approach

◆ Techniques Used

- Interviews with experts
- Observation
- Case study analysis
- Machine learning algorithms
- Data mining

◆ Challenges in Knowledge Acquisition

1. Knowledge Acquisition Bottleneck

- Difficult to extract knowledge from experts

2. Incomplete Knowledge

- Missing or partial data

3. Changing Knowledge

- Knowledge becomes outdated over time

4. Representation Issues

- Hard to convert human knowledge into machine-readable form

◆ Importance of Knowledge Acquisition

- Builds the **knowledge base** of AI systems
- Improves **decision-making**
- Helps in **expert systems, chatbots, recommendation systems**
- Makes AI **more intelligent and accurate**

📌 Short Exam Definition

Knowledge Acquisition is the process of gathering, organizing, and storing knowledge from various sources to build a knowledge base for an AI system.

What is Early Machine Learning?



What is Early Machine Learning?

Early Machine Learning refers to the **initial approaches and techniques** developed in AI to enable computers to **learn from data and experience**, mainly during the **1950s–1980s**.

👉 Focus was on:

- Simple models
- Rule-based learning
- Limited data and computing power

◆ Key Characteristics

- Based on **symbolic reasoning**
- Used **small datasets**
- Relied heavily on **human guidance**
- Focus on **learning rules and patterns**

◆ Important Early Approaches

1. Rote Learning

- System **memorizes facts and results**
- No generalization

👉 Example: Storing previous solutions and reusing them

2. Learning by Deduction

- Uses **logical reasoning** to derive new knowledge
- Based on already known facts

👉 Example:

If $A \rightarrow B$ and $B \rightarrow C \Rightarrow A \rightarrow C$

3. Learning by Induction

- Learning **general rules from specific examples**

👉 Example:

Observing many birds fly → conclude "Birds can fly"

4. Decision Tree Learning

- Learning decisions in **tree structure form**
- Nodes = decisions, branches = outcomes

👉 Used in early classification problems

5. Neural Network (Early Models)

- Inspired by the human brain
- Basic models like **Perceptron**

👉 Could perform simple classification tasks

♦ Famous Early System

♦ Samuel Checkers Program

- Developed by Arthur Samuel
 - One of the first programs to **learn from experience**
 - Improved its performance by playing repeatedly
-

♦ Limitations of Early Machine Learning

- Limited **computing power**
 - Lack of **large datasets**
 - Poor performance on **complex problems**
 - Over-reliance on **manual rule creation**
-

♦ Importance

- Laid the **foundation of modern Machine Learning**
 - Introduced key concepts like:
 - Learning from data
 - Generalization
 - Pattern recognition
-

Short Exam Definition

Early Machine Learning refers to the initial methods developed in AI (1950s–1980s) that enabled systems to learn from data using simple models, rules, and logical reasoning.

What is Learning by Induction?



What is Learning by Induction?

Learning by Induction is a method in AI where a system **learns general rules or patterns from specific examples**.

👉 In simple words:

It means **"learning from examples"**

◆ Key Idea

- Start with **specific observations (data)**
- Derive a **general rule (hypothesis)**



Example:

- Given:
 - Crow → flies
 - Sparrow → flies
 - Pigeon → flies

👉 Induced Rule:

"All birds can fly" (generalization)

◆ Steps in Inductive Learning

1. **Collect Examples (Training Data)**
 2. **Analyze Patterns**
 3. **Form Hypothesis (General Rule)**
 4. **Test the Rule on New Data**
-

◆ Characteristics

- Data-driven approach
 - Produces **generalized knowledge**
 - May not always be 100% correct
 - Handles **uncertainty**
-

◆ Types of Inductive Learning

1. Supervised Inductive Learning

- Uses **labeled data**
- Learns mapping from input → output

👉 Example: Classification problems

2. Unsupervised Inductive Learning

- Uses **unlabeled data**
- Finds hidden patterns

👉 Example: Clustering

♦ Applications

- Spam email detection
 - Medical diagnosis
 - Recommendation systems
 - Pattern recognition
-

♦ Advantages

- Can **generalize new cases**
 - Useful for **real-world problems**
 - Learns automatically from data
-

♦ Limitations

- May produce **incorrect generalizations**
 - Depends on **quality of data**
 - Requires sufficient training data
-

♦ Relation to Machine Learning

👉 Most modern ML algorithms (like decision trees, neural networks) are based on **inductive learning**.

📌 Short Exam Definition

Learning by Induction is the process of deriving general rules from specific examples or data in order to make predictions or decisions.

What is an Expert System?



What is an Expert System?

An **Expert System** is a computer program in AI that **emulates the decision-making ability of a human expert** in a specific domain.

👉 In simple words:

It is a system that **thinks and gives advice like an expert**

◆ Key Idea

- Uses **stored knowledge + reasoning**
 - Solves **complex problems**
 - Provides **decisions, suggestions, or diagnosis**
-

◆ Basic Components of Expert System

1. Knowledge Base

- Contains:
 - Facts
 - Rules (IF–THEN)

👉 Example:

IF fever AND cough → Possible flu

2. Inference Engine

- The “brain” of the system
 - Applies rules to data to derive conclusions
-

3. User Interface

- Allows user to interact with the system
 - Input queries and get results
-

4. Explanation Facility

- Explains **how and why** a decision was made
-

◆ Working of Expert System

1. User provides input (problem)
 2. Inference engine checks rules in knowledge base
 3. Applies logic
 4. Produces result (advice/decision)
-

◆ Famous Example

◆ MYCIN

- Developed for **medical diagnosis**
 - Suggested antibiotics based on symptoms
-

◆ Characteristics

- Domain-specific (e.g., medical, finance)
 - High performance in limited area
 - Consistent decision-making
 - Uses symbolic reasoning
-

◆ Advantages

- Available **24/7**
 - Reduces human error
 - Fast decision-making
 - Preserves expert knowledge
-

◆ Limitations

- Limited to specific domain
 - Expensive to build
 - Cannot handle unexpected situations well
 - Lacks human intuition
-

◆ Applications

- Medical diagnosis

- Banking & finance
- Agriculture
- Fault diagnosis in machines

What are Phases of an Expert System?



What are Phases of an Expert System?

The **Phases of an Expert System** refer to the **steps involved in developing and implementing** an expert system.

👉 In simple words:

It is the **lifecycle of building an expert system**

◆ Main Phases of Expert System

◆ 1. Problem Identification

- Define the **problem domain**
- Identify goals and scope

👉 Example: Medical diagnosis, fault detection

◆ 2. Knowledge Acquisition

- Collect knowledge from:
 - Experts
 - Books
 - Databases

👉 Most difficult phase (knowledge acquisition bottleneck)

◆ 3. Knowledge Representation

- Convert knowledge into **machine-readable form**
- Use:
 - Rules (IF-THEN)
 - Frames
 - Semantic networks

◆ 4. System Design (Development)

- Build components:
 - Knowledge Base
 - Inference Engine

- User Interface
-

◆ 5. Implementation

- Code and develop the system
 - Integrate all modules
-

◆ 6. Testing & Validation

- Check:
 - Accuracy
 - Performance
 - Correctness

👉 Compare with expert decisions

◆ 7. Deployment

- Make system available to users
 - Real-world usage begins
-

◆ 8. Maintenance & Update

- Update knowledge base
- Improve system over time

👉 Important because knowledge changes

◆ Diagram (Conceptual Flow)

Problem → Knowledge Collection → Representation → Development → Testing → Deployment → Maintenance

◆ Key Point

👉 The most critical phase is:

Knowledge Acquisition (hard to extract expert knowledge)

📌 Short Exam Definition

Phases of an Expert System are the steps involved in its development, including problem identification, knowledge acquisition, representation,

system design, implementation, testing, deployment, and maintenance.

Quick Memory Trick

👉 **P K K S I T D M**

(Problem → Knowledge → Knowledge → System → Implementation →
Testing → Deployment → Maintenance)

What are Characteristics of an Expert System?



What are Characteristics of an Expert System?

Characteristics of an Expert System are the **key features that make it behave like a human expert** in solving problems within a specific domain.

◆ Main Characteristics

◆ 1. Domain-Specific

- Works in a **specific area only**
- Not a general-purpose system

👉 Example: Medical diagnosis, finance

◆ 2. High Performance

- Provides **accurate and efficient solutions**
- Comparable to human experts

◆ 3. Knowledge-Based

- Relies on a **knowledge base**
- Uses:
 - Facts
 - Rules (IF-THEN)

◆ 4. Reasoning Capability

- Uses an **inference engine**
- Applies logic to derive conclusions

◆ 5. Explanation Facility

- Can explain:
 - **How** a decision was made
 - **Why** it was made

◆ 6. Consistency

- Produces **same results for same input**

- No mood or fatigue like humans
-

♦ 7. Reliability

- Dependable for decision-making
 - Reduces human error
-

♦ 8. Fast Response

- Provides **quick solutions**
 - Saves time
-

♦ 9. User-Friendly Interface

- Easy interaction between:
 - User
 - System
-

♦ 10. Ability to Handle Complex Problems

- Solves problems that require **expert-level knowledge**
-

♦ Summary in One Line

👉 Expert systems are **domain-specific, knowledge-based systems with reasoning ability, consistency, and explanation features.**

📌 Short Exam Definition

An Expert System is characterized by domain-specific knowledge, high performance, reasoning capability, consistency, explanation facility, and fast, reliable decision-making.

🎯 Quick Memory Trick

👉 **D H K R E C R F U C**

(Domain, High performance, Knowledge, Reasoning, Explanation, Consistency, Reliability, Fast, User-friendly, Complex problem solving)

What is Executive Support System (ESS)?



What is Executive Support System (ESS)?

An **Executive Support System (ESS)** is an information system designed to help **top-level managers (executives)** make **strategic decisions** by providing **summarized, critical, and real-time information**.

👉 In simple words:

It helps **CEOs, directors, and senior managers** take **important business decisions**

◆ Key Features of ESS

- Provides **high-level summarized data**
- Supports **strategic decision-making**
- Uses **graphs, dashboards, and reports**
- Access to both **internal and external data**
- Easy-to-use interface

◆ Components of ESS

◆ 1. Data Management

- Collects data from:
 - Internal systems (sales, finance)
 - External sources (market trends)

◆ 2. Model Management

- Uses analytical tools for:
 - Forecasting
 - Decision-making

◆ 3. User Interface

- Simple dashboards and reports
- Easy navigation for executives

◆ Functions of ESS

- Strategic planning
 - Performance monitoring
 - Trend analysis
 - Forecasting future outcomes
-

◆ Characteristics

- Focuses on **long-term decisions**
 - Handles **unstructured problems**
 - Provides **quick access to critical information**
 - Flexible and interactive
-

◆ Advantages

- Improves **decision quality**
 - Saves time for executives
 - Provides **competitive advantage**
 - Better understanding of business trends
-

◆ Limitations

- Expensive to develop
 - Depends on data accuracy
 - Limited to top-level management
 - Requires regular updates
-

◆ Example

👉 A company dashboard showing:

- Sales performance
- Profit trends
- Market growth

Helps executives decide:

- Whether to expand business
 - Launch new products
-

Short Exam Definition

An Executive Support System (ESS) is a computer-based system that provides top-level management with summarized information and analytical tools to support strategic decision-making.

What is Decision Support System (DSS)?



What is Decision Support System (DSS)?

A **Decision Support System (DSS)** is a computer-based system that helps **managers and decision-makers** make **better decisions** by analyzing data, models, and information.

👉 In simple words:

It helps people **analyze problems and choose the best option**

◆ Key Idea

- Supports **decision-making**, not replaces humans
- Works with **data + models + user input**
- Used for **semi-structured and unstructured problems**

◆ Components of DSS

◆ 1. Data Management System

- Stores and manages data
- Sources:
 - Databases
 - External data

◆ 2. Model Management System

- Contains mathematical and analytical models
- Used for:
 - Forecasting
 - Simulation
 - Optimization

◆ 3. User Interface

- Interaction between user and system
- Dashboards, reports, queries

◆ 4. Knowledge Base (Optional)

- Stores rules and expertise
-

◆ Types of DSS

1. Data-Driven DSS

- Focus on large datasets

2. Model-Driven DSS

- Uses mathematical models

3. Knowledge-Driven DSS

- Uses expert knowledge

4. Document-Driven DSS

- Uses documents and reports

5. Communication-Driven DSS

- Supports group decision-making
-

◆ Characteristics

- Interactive and flexible
 - Supports decision-making at different levels
 - Helps in **what-if analysis**
 - User-friendly
-

◆ Advantages

- Improves decision quality
 - Saves time
 - Helps analyze complex problems
 - Supports quick decision-making
-

◆ Limitations

- Depends on data quality
- Costly to develop

- Requires skilled users
 - Cannot replace human judgment
-

◆ Example

👉 A business system that:

- Analyzes sales data
 - Predicts future demand
 - Suggests best pricing strategy
-

◆ Difference from ESS

- **DSS** → Used by middle-level managers (tactical decisions)
- **ESS** → Used by top-level executives (strategic decisions)

What are Applications of Expert Systems?



What are Applications of Expert Systems?

Applications of Expert Systems refer to the **real-world areas where expert systems are used** to solve problems and provide expert-level decisions.

👉 In simple words:

Wherever **expert advice or decision-making is needed**, expert systems can be applied.

◆ Major Application Areas

◆ 1. Medical Field

- Disease diagnosis
- Suggesting treatments
- Analyzing patient data

👉 Example: MYCIN

◆ 2. Finance & Banking

- Loan approval
- Risk analysis
- Fraud detection

◆ 3. Agriculture

- Crop disease diagnosis
- Fertilizer recommendation
- Weather-based advice

◆ 4. Manufacturing

- Fault detection in machines
- Quality control
- Process optimization

◆ 5. Education

- Intelligent tutoring systems

- Personalized learning guidance
-

◆ 6. Business & Management

- Decision-making support
 - Strategic planning
-

◆ 7. Engineering

- Design systems
 - Troubleshooting technical issues
-

◆ 8. Customer Support

- Chatbots and helpdesk systems
 - Automated query resolution
-

◆ 9. Military & Defense

- Strategy planning
 - Threat analysis
-

◆ 10. Transportation

- Route planning
 - Traffic management
-

◆ Key Idea

👉 Expert systems are used in **any domain where human expertise is required repeatedly**.

◆ Advantages in Applications

- Saves time and cost
 - Provides consistent decisions
 - Available 24/7
 - Reduces dependency on human experts
-

📌 Short Exam Definition

Expert Systems are applied in various fields such as medical, finance, agriculture, manufacturing, and education to provide expert-level decision-making and problem-solving.

Module V: Robotics and its Applications

What is Robotics?



What is Robotics?

Robotics is a branch of Artificial Intelligence and engineering that deals with the **design, construction, operation, and use of robots**.

👉 In simple words:

Robotics is about **creating machines (robots) that can perform tasks automatically or with minimal human effort**.

♦ What is a Robot?

A **robot** is a programmable machine that can:

- Sense environment
- Process information
- Take actions

👉 Example: Industrial robot arms, cleaning robots

♦ Components of a Robot

♦ 1. Sensors

- Collect data from environment
- 👉 Example: Camera, temperature sensor
-

♦ 2. Actuators

- Help in movement and action
- 👉 Example: Motors, wheels
-

♦ 3. Controller

- "Brain" of the robot
 - Processes input and gives commands
-

♦ 4. Power Supply

- Provides energy to robot
-

♦ Types of Robots

1. **Industrial Robots** – Used in factories

2. **Service Robots** – Used in homes and offices
 3. **Mobile Robots** – Move from place to place
 4. **Humanoid Robots** – Look and act like humans
-

◆ Applications of Robotics

◆ 1. Manufacturing Industry

- Assembly lines
- Welding, painting

👉 Improves speed and accuracy

◆ 2. Medical Field

- Robotic surgery
- Patient care

👉 Example: Robot-assisted surgery

◆ 3. Agriculture

- Automated harvesting
 - Crop monitoring
-

◆ 4. Military & Defense

- Bomb disposal robots
 - Surveillance
-

◆ 5. Space Exploration

- Planet exploration
- 👉 Example: Mars Rover
-

◆ 6. Domestic Use

- Cleaning robots
- 👉 Example: Roomba
-

◆ 7. Transportation

- Self-driving cars
 - Automated delivery systems
-

◆ 8. Education

- Teaching and research
-

◆ Advantages of Robotics

- High accuracy and precision
 - Works in dangerous environments
 - Reduces human effort
 - Increases productivity
-

◆ Limitations

- High cost
- Requires maintenance
- Limited decision-making ability
- Job displacement concerns

What is DDD Concept?



What is DDD Concept?

DDD (Sense–Plan–Act / Deliberate–Decide–Do) is a concept in robotics that describes **how a robot performs tasks step by step**.

👉 In simple words:

It explains **how a robot thinks and acts**

♦ Full Form of DDD

- **D – Deliberate (Sense/Perceive)**
- **D – Decide (Plan)**
- **D – Do (Act)**

♦ Steps in DDD Concept

♦ 1. Deliberate (Sense)

- Robot **collects information** from environment
- Uses sensors

👉 Example: Camera detects obstacles

♦ 2. Decide (Plan)

- Robot **processes information**
- Makes decision using logic/algorithms

👉 Example: Finds best path to avoid obstacle

♦ 3. Do (Act)

- Robot **performs action** using actuators

👉 Example: Moves left or right

♦ Diagram Flow

👉 Sense → Decide → Act

♦ Example

👉 Self-driving car:

- Sense → Detects road, vehicles
 - Decide → Chooses safe route
 - Act → Moves steering, brakes
-

♦ **Characteristics**

- Sequential process
 - Based on perception and action
 - Widely used in robotics and AI systems
-

♦ **Advantages**

- Simple and easy to understand
 - Logical decision-making process
 - Useful in real-world robotics
-

♦ **Limitations**

- Time-consuming (needs full planning)
- Not suitable for highly dynamic environments
- Depends on accurate sensing

What are Intelligent Robots?



What are Intelligent Robots?

Intelligent Robots are robots that can **sense, think, learn, and act intelligently** using techniques from Artificial Intelligence.

👉 In simple words:

They are robots that can **make decisions on their own like humans**

◆ Key Features of Intelligent Robots

- **Perception** → Understand environment using sensors
- **Reasoning** → Make decisions using logic
- **Learning** → Improve performance over time
- **Autonomy** → Work without human intervention

◆ Components of Intelligent Robots

◆ 1. Sensors

- Collect data (vision, sound, temperature)

◆ 2. Processing Unit (AI Brain)

- Uses AI algorithms to analyze data

◆ 3. Knowledge Base

- Stores learned information

◆ 4. Actuators

- Perform actions (movement, tasks)

◆ Capabilities

- Object recognition
 - Path planning
 - Decision-making
 - Problem-solving
 - Adaptation to environment
-

◆ Examples of Intelligent Robots

- Self-driving cars
- Humanoid robots
- Service robots

👉 Example: Sophia

◆ Applications

- Healthcare (surgery robots)
 - Industry (automation)
 - Military (surveillance robots)
 - Home (cleaning robots)
 - Space exploration
-

◆ Advantages

- Works in dangerous environments
 - High accuracy
 - Reduces human effort
 - Learns and improves
-

◆ Limitations

- Expensive
- Complex design
- Ethical concerns
- Limited human-like understanding

What is Robot Anatomy?



What is Robot Anatomy?

Robot Anatomy refers to the **physical structure and components of a robot**, including how its parts are arranged and connected to perform tasks.

👉 In simple words:

It is the **body structure of a robot**

◆ Main Parts of Robot Anatomy

◆ 1. Manipulator (Arm)

- Main body or **mechanical arm** of the robot
- Used for movement and task execution

👉 Similar to human arm

◆ 2. End Effector

- Tool attached at the end of the arm
- Interacts with objects

👉 Examples:

- Gripper
- Welding tool

◆ 3. Joints

- Connect different parts of the robot
- Allow movement

👉 Types:

- Rotary (circular movement)
- Linear (straight movement)

◆ 4. Links

- Rigid parts between joints
 - Form the structure of the robot
-

♦ 5. Actuators

- Provide motion and force

👉 Examples:

- Electric motors
 - Hydraulic systems
-

♦ 6. Sensors

- Detect environment and internal state

👉 Examples:

- Camera
 - Temperature sensor
-

♦ 7. Controller

- "Brain" of the robot
 - Controls all operations
-

♦ 8. Power Supply

- Provides energy (battery/electricity)
-

♦ Degrees of Freedom (DOF)

- Number of independent movements a robot can make

👉 Example:

More DOF → More flexibility

♦ Types of Robot Configurations

1. Cartesian Robot
2. Cylindrical Robot
3. Spherical Robot
4. Articulated Robot

What are Laws of Robotics?



What are Laws of Robotics?

The **Laws of Robotics** are a set of rules designed to ensure that **robots behave safely and ethically with humans**.

👉 These laws were introduced by Isaac Asimov in his science fiction works.

♦ The Three Laws of Robotics

♦ 1. First Law

👉 **A robot may not injure a human being or, through inaction, allow a human being to come to harm.**

✓ Highest priority

✓ Human safety comes first

♦ 2. Second Law

👉 **A robot must obey the orders given by human beings, except where such orders would conflict with the First Law.**

✓ Follow human instructions

✓ But not if it harms humans

♦ 3. Third Law

👉 **A robot must protect its own existence as long as such protection does not conflict with the First or Second Law.**

✓ Self-protection

✓ Lowest priority

♦ Priority Order

1 First Law (Human safety)

2 Second Law (Obedience)

3 Third Law (Self-protection)

♦ Later Addition (Zeroth Law)

👉 Added later by Isaac Asimov:

A robot may not harm humanity, or, by inaction, allow humanity to come to harm.

✓ Even higher than First Law

✓ Focus on humanity as a whole

◆ Importance

- Ensures **safe interaction between humans and robots**
 - Provides **ethical guidelines**
 - Influences modern AI ethics
-

◆ Limitations

- Theoretical (not practically implemented fully)
- Hard to apply in real-world complex situations
- Ambiguity in interpretation

What is History of Robotics?



What is History of Robotics?

The **History of Robotics** describes the **development and evolution of robots** from early mechanical devices to modern intelligent systems.

👉 In simple words:

It shows **how robots evolved over time**

◆ Early Foundations (Ancient Times)

- Early ideas of automated machines in **ancient civilizations**
- Mechanical devices created using simple mechanisms

👉 Example: Water clocks, automatic doors

◆ 1. 1920 – Origin of the Word “Robot”

- The word “**Robot**” was introduced by Karel Čapek
- In his play R.U.R. (Rossum’s Universal Robots)

👉 “Robot” means **forced labor**

◆ 2. 1940s–1950s – Foundations of Robotics

- Introduction of **robotic concepts and AI ideas**
 - Isaac Asimov proposed **Laws of Robotics**
-

◆ 3. 1961 – First Industrial Robot

- Unimate was developed
- Used in **General Motors factory**

👉 Beginning of industrial automation

◆ 4. 1970s–1980s – Growth of Industrial Robots

- Robots used in:
 - Welding
 - Assembly
 - Manufacturing

👉 Increased productivity

◆ 5. 1990s – Intelligent Robotics

- Integration of:
 - Sensors
 - AI techniques

👉 Robots became more **flexible and adaptive**

◆ 6. 2000s – Modern Robotics

- Development of:
 - Humanoid robots
 - Service robots

👉 Example: ASIMO

◆ 7. Present & Future

- AI-powered robots
- Self-driving cars
- Robots in healthcare, space, and homes

👉 Example: Sophia

◆ Key Milestones Summary

Year	Development
1920	Term "Robot" introduced
1950s	AI & robotics concepts
1961	First industrial robot
1980s	Industrial growth
2000s	Intelligent robots
Present	AI-based robotics

◆ Importance

- Shows evolution of technology

- Helps understand modern robotics
- Foundation for future innovations

What is Robotics Terminology?



What is Robotics Terminology?

Robotics Terminology refers to the **basic terms and concepts used in robotics** to understand how robots are designed, controlled, and operated.

👉 In simple words:

It is the **vocabulary of robotics**

♦ Important Terms in Robotics

♦ 1. Robot

- A **programmable machine** that performs tasks automatically

♦ 2. Manipulator

- Mechanical arm used to perform tasks

♦ 3. End Effector

- Tool attached at the end of the robot arm
👉 Example: Gripper, welding tool

♦ 4. Actuator

- Device that **produces motion**
👉 Example: Motor, hydraulic system

♦ 5. Sensor

- Detects environment
👉 Example: Camera, infrared sensor

♦ 6. Degrees of Freedom (DOF)

- Number of **independent movements** a robot can make

♦ 7. Workspace

- Area where the robot can operate

♦ 8. Payload

- Maximum weight a robot can carry
-

♦ 9. Resolution

- Smallest movement a robot can make
-

♦ 10. Accuracy

- How close the robot reaches the desired position
-

♦ 11. Repeatability

- Ability to return to the same position repeatedly
-

♦ 12. Control System

- System that controls robot operations
-

♦ 13. Trajectory

- Path followed by a robot
-

♦ 14. Automation

- Use of machines to perform tasks with minimal human effort
-

♦ 15. Artificial Intelligence (AI)

- Enables robots to **learn and make decisions**
👉 Artificial Intelligence
-

♦ Importance of Terminology

- Helps in **understanding robotics concepts**
- Essential for **designing and programming robots**
- Useful in **technical communication**

What is Accuracy & Repeatability in Robotics?



What is Accuracy & Repeatability in Robotics?

These are **performance measures of a robot** that describe how precisely and consistently it can perform tasks.

♦ 1. Accuracy

✓ Definition:

Accuracy is the ability of a robot to reach the **exact desired position**.

👉 In simple words:

How close the robot gets to the **target point**

📌 Example:

- Target position = (10, 10)
- Robot reaches = (9.8, 10.2)

👉 Small error → High accuracy

♦ Key Point:

- Depends on calibration and system design
 - Affected by errors (mechanical, sensor, etc.)
-

♦ 2. Repeatability

✓ Definition:

Repeatability is the ability of a robot to **return to the same position again and again**.

👉 In simple words:

How consistent the robot is

📌 Example:

- Robot reaches (9.8, 10.2) every time
 - 👉 Even if slightly off target → High repeatability
-

♦ Key Point:

- More important in industrial robots
- Ensures consistent performance

◆ Difference Between Accuracy & Repeatability

Feature	Accuracy	Repeatability
Meaning	Closeness to target	Consistency
Focus	Correct position	Same position repeatedly
Importance	Precision tasks	Industrial automation
Example	Hits exact point	Hits same wrong point every time

◆ Visual Understanding

👉 Think of a dartboard:

- 🎯 **High Accuracy + High Repeatability** → All darts hit center
- 🎯 **Low Accuracy + High Repeatability** → All darts hit same wrong spot
- 🎯 **High Accuracy + Low Repeatability** → Darts near center but scattered

◆ Importance

- Ensures **quality and precision**
- Important in:
 - Manufacturing
 - Assembly
 - Robotics automation

What are Robot Specifications?



What are Robot Specifications?

Robot Specifications are the **technical details or parameters** that define a robot's **performance, capability, and limitations**.

👉 In simple words:

They describe **what a robot can do and how well it can do it**

♦ Important Robot Specifications

♦ 1. Payload

- Maximum weight a robot can carry

♦ 2. Reach (Workspace)

- Maximum distance the robot arm can extend

♦ 3. Degrees of Freedom (DOF)

- Number of independent movements

♦ 4. Accuracy

- How close the robot reaches the target

♦ 5. Repeatability

- Ability to return to same position

♦ 6. Resolution

- Smallest movement robot can detect or make

♦ 7. Speed

- How fast the robot performs tasks

♦ What is Robot Speed?

Robot Speed refers to how quickly a robot can **move or complete a task**.

👉 Measured in:

- m/s (meters per second)
- mm/s

- degrees/sec (for joints)
-

◆ Types of Robot Speed

◆ 1. Joint Speed

- Speed of individual joints
-

◆ 2. End Effector Speed

- Speed of the tool (gripper, etc.)
-

◆ 3. Cycle Time

- Time taken to complete one full task

👉 Important in industries

◆ Factors Affecting Speed

- Payload (more weight → slower speed)
 - Robot design
 - Power of actuators
 - Control system
-

◆ Importance of Speed

- Increases productivity
 - Reduces operation time
 - Improves efficiency in industries
-

◆ Trade-off

👉 High speed may reduce:

- Accuracy
- Safety

👉 So balance is required

What are Robot Joints and Links?



What are Robot Joints and Links?

- **Joints and Links** are the **basic structural elements of a robot** that enable movement and flexibility.

👉 In simple words:

- **Links** = rigid parts (like bones)
- **Joints** = connections that allow movement (like human joints)

♦ 1. Robot Links

✅ Definition:

Links are the **rigid segments** of a robot that connect joints and form the robot's structure.

📌 Features:

- Provide **structure and support**
- Do not move independently
- Connected through joints

📌 Example:

- Robot arm segments

♦ 2. Robot Joints

✅ Definition:

Joints are the **connections between links** that allow motion.

📌 Features:

- Enable movement
- Define robot flexibility
- Determine Degrees of Freedom (DOF)

♦ Types of Robot Joints

♦ 1. Revolute Joint (R)

- Rotational movement

- Moves like a hinge

👉 Example: Human elbow

♦ 2. Prismatic Joint (P)

- Linear (sliding) movement

👉 Example: Sliding drawer

♦ 3. Cylindrical Joint (C)

- Combination of:
 - Rotation + linear movement
-

♦ 4. Spherical Joint (S)

- Movement in multiple directions

👉 Example: Human shoulder

♦ 5. Helical Joint (H)

- Screw-like motion (rotation + translation)
-

♦ Relationship Between Joints & Links

👉 Links are connected by joints to form a **kinematic chain**

👉 More joints → More flexibility (higher DOF)

♦ Example

👉 Robot arm:

- Links → Arm segments
 - Joints → Elbow, shoulder connections
-

♦ Importance

- Determines robot movement
- Affects flexibility and reach
- Essential for robot design

What are Types of Robots?



What are Types of Robots?

Types of Robots are classifications based on their **structure, movement, and application**.

👉 In simple words:

Different robots are designed for **different tasks and environments**

◆ Classification of Robots

◆ 1. Based on Structure (Geometry)

◆ a) Cartesian Robot

- Moves in **straight lines (X, Y, Z axes)**
- Simple and accurate

👉 Used in CNC machines

◆ b) Cylindrical Robot

- Combines **rotational + linear motion**
- Cylindrical workspace

◆ c) Spherical (Polar) Robot

- Movement in **spherical coordinates**
- Flexible reach

◆ d) Articulated Robot

- Has multiple rotary joints
- Similar to human arm

👉 Most commonly used in industries

◆ 2. Based on Application

◆ a) Industrial Robots

- Used in manufacturing
 - 👉 Welding, assembly

◆ b) Service Robots

- Used for personal or professional services

👉 Example: Roomba

◆ c) Medical Robots

- Used in healthcare
 - 👉 Surgery assistance
-

◆ d) Military Robots

- Used in defense
 - 👉 Bomb disposal
-

◆ e) Space Robots

- Used in space missions
 - 👉 Example: Mars Rover
-

◆ 3. Based on Mobility

◆ a) Fixed Robots

- Stationary
 - Used in factories
-

◆ b) Mobile Robots

- Can move around
 - 👉 Examples:
 - Self-driving cars
 - Delivery robots
-

◆ 4. Based on Control System

◆ a) Autonomous Robots

- Work independently
-

◆ b) Semi-Autonomous Robots

- Partially controlled by humans
-

◆ c) Teleoperated Robots

- Fully controlled by humans remotely
-

◆ Summary

👉 Robots can be classified based on:

- Structure
- Application
- Mobility
- Control system

What is System Architecture in Robotics?



What is System Architecture in Robotics?

System Architecture refers to the **overall structure and organization of components** in a robotic system, and how they **interact to perform tasks**.

👉 In simple words:

It is the **blueprint of how a robot works internally**

◆ Basic Architecture of a Robot

👉 Input → Processing → Output

1. **Sensors (Input)**
 2. **Controller / Processor (Brain)**
 3. **Actuators (Output)**
-

◆ Components of System Architecture

◆ 1. Sensors

- Collect data from environment
 - 👉 Example: Camera, infrared sensor
-

◆ 2. Control System

- Processes input data
 - Makes decisions using algorithms
-

◆ 3. Actuators

- Execute actions
 - 👉 Example: Motors, wheels
-

◆ 4. Power System

- Supplies energy
-

◆ 5. Communication System

- Enables interaction between components
 - Can include wireless communication
-

◆ Types of System Architecture

◆ 1. Centralized Architecture

- One central controller controls everything

👉 ✅ Simple

👉 ❌ Single point of failure

◆ 2. Decentralized Architecture

- Multiple controllers share tasks

👉 ✅ More reliable

👉 ❌ Complex

◆ 3. Hierarchical Architecture

- Organized in levels:
 - High-level (planning)
 - Mid-level (control)
 - Low-level (execution)

👉 Most commonly used

◆ 4. Reactive Architecture

- Direct response to environment
- No complex planning

👉 Fast but less intelligent

◆ Working Flow

👉 Sensors → Controller → Decision → Actuators → Action

◆ Importance

- Helps design efficient robots
- Improves performance and reliability
- Essential for complex robotic systems

What are Hydraulic, Pneumatic & Electric Systems?



What are Hydraulic, Pneumatic & Electric Systems?

These are **actuation systems** used in robots to **produce movement and perform tasks**.

👉 In simple words:

They are different ways to **power and move a robot**

♦ 1. Hydraulic System

✅ Definition:

A **hydraulic system** uses **liquid (usually oil)** under pressure to generate motion.

♦ Features:

- High force and power
 - Smooth operation
-

♦ Advantages:

- Can lift heavy loads
 - High strength
-

♦ Limitations:

- Expensive
 - Leakage issues
 - Requires maintenance
-

📌 Applications:

- Heavy machinery
 - Industrial robots
-

♦ 2. Pneumatic System

✅ Definition:

A **pneumatic system** uses **compressed air** to produce motion.

◆ **Features:**

- Fast operation
 - Simple design
-

◆ **Advantages:**

- Low cost
 - Easy maintenance
 - Safe (no fire hazard)
-

◆ **Limitations:**

- Less power than hydraulic
 - Less precise
-

 Applications:

- Pick-and-place robots
 - Packaging systems
-

◆ **3. Electric System**

 Definition:

An **electric system** uses **electric motors and power** to control movement.

◆ **Features:**

- High precision
 - Easy control
-

◆ **Advantages:**

- Accurate and reliable
 - Clean and efficient
 - Widely used
-

◆ **Limitations:**

- Limited power for heavy loads

- Depends on electricity
-

Applications:

- Industrial robots
- Service robots
- Home automation

What are Neural Networks?



What are Neural Networks?

Neural Networks are a type of AI model inspired by the **human brain**, used to recognize patterns, learn from data, and make decisions.

👉 They are a core concept in Artificial Intelligence and Machine Learning.

👉 In simple words:

A neural network is a system of **connected nodes (neurons)** that learn from data.

◆ Basic Structure of Neural Network

A neural network consists of **layers of neurons**:

◆ 1. Input Layer

- Receives input data

◆ 2. Hidden Layer(s)

- Processes data using weights and activation functions

◆ 3. Output Layer

- Produces final result

◆ Working of Neural Network

1. Input data is fed into the network
2. Each neuron applies:
 - Weight
 - Activation function
3. Data passes through layers
4. Output is generated
5. Model learns by adjusting weights

◆ Key Components

- **Neuron (Node)** → Basic unit
- **Weights** → Importance of input

- **Bias** → Extra parameter
 - **Activation Function** → Decides output
-

◆ **Types of Neural Networks**

- **Feedforward Neural Network**
 - **Recurrent Neural Network (RNN)**
 - **Convolutional Neural Network (CNN)**
-

◆ **Applications**

- Image recognition
 - Speech recognition
 - Chatbots
 - Recommendation systems
-

◆ **Advantages**

- Learns complex patterns
 - High accuracy
 - Handles large data
-

◆ **Limitations**

- Requires large data
- Computationally expensive
- Hard to interpret

What is Fuzzy Logic?



What is Fuzzy Logic?

Fuzzy Logic is a method in Artificial Intelligence that allows systems to make decisions based on **degrees of truth rather than exact true/false values**.

👉 In simple words:

Instead of only **0 (false) or 1 (true)**, fuzzy logic works with **values in between (like 0.2, 0.7, etc.)**

◆ Key Idea

👉 Classical Logic:

- TRUE or FALSE (0 or 1)

👉 Fuzzy Logic:

- PARTIALLY TRUE (between 0 and 1)

◆ Example

👉 Temperature:

- Classical Logic:
 - Cold OR Hot
- Fuzzy Logic:
 - $30^{\circ}\text{C} = 0.6 \text{ Hot} + 0.4 \text{ Warm}$

◆ Components of Fuzzy Logic System

◆ 1. Fuzzification

- Convert crisp input into fuzzy values

◆ 2. Knowledge Base

- Contains rules and membership functions

◆ 3. Inference Engine

- Applies fuzzy rules

◆ 4. Defuzzification

- Convert fuzzy output into crisp value
-

◆ **Working Steps**

1. Input data (crisp value)
 2. Fuzzification
 3. Apply rules
 4. Inference
 5. Defuzzification
 6. Output
-

◆ **Applications**

- Washing machines
 - Air conditioners
 - Traffic control systems
 - Medical diagnosis
-

◆ **Advantages**

- Handles uncertainty
 - Flexible and human-like reasoning
 - Easy to understand
-

◆ **Limitations**

- Not always precise
 - Rule design can be complex
 - Depends on expert knowledge
-

- 5.6.3 LISP and Prolog

- 5.6.4 Soft Computing Research
- 5.6.5 Knowledge Management
- 5.6.6 Ontology
- 5.6.7 Natural Language Processing (NLP)
- 5.6.8 Genetic Algorithm

